Faculty of Computers and Information,
Assiut University

Money Mirror

Software Requirements Specifications

Under the supervision of:
Dr. Marwa Hussien

Students:
1. Jannah Ayman Abdelraouf (CS)
2. Nancy Saad Muhammad (CS)
3. Rawan Sotohy Mohammed (CS)
4. Arwa Hassan Mohammed (IS)
5. Rahma Tarek Mohammed (IS)
6. Doha Emad Abdeltawab (IS)

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) provides a detailed description of the requirements for the Money Mirror system. It gives a complete understanding of the Money Mirror mobile application, which assists parents in monitoring their children's spending habits, understanding their financial personality types through AI-driven insights, and fostering financial literacy in children in an engaging, age-appropriate manner.

1.2 Intended Audience and Reading Suggestions

The intended audience includes the project development team, parents seeking to guide their children's financial decisions, and guardians.
Reading suggestions:

1. Read Section 1.3 Product Scope for a high-level overview of the app's goals and boundaries.
2. Skim Section 2.2 Product Functions for a summary of core capabilities.
3. Dive into Section 4.3 Detailed Use Cases for parent/child interaction scenarios.
4. Review Section 3 (External Interface Requirements) and Section 6 (Nonfunctional Requirements) for technical and quality details.

1.3 Product Scope

Money Mirror is a mobile solution designed to improve children's financial literacy and allow parents to monitor spending behavior. The system adopts a hybrid architecture:

- Story quizzes are pre-generated offline in batches using a Large Language Model (Claude 3.5 Sonnet), validated, and stored in the database. No AI generation occurs during user interaction.
- Personality profiling and behavioral scoring use a deterministic, rule-based engine. A transition to trained ML models is planned once sufficient behavioral data is collected.
- A weekly background job aggregates behavior data and quiz results to evolve the child's personality profile over time.
- An AI chatbot (for both children and parents) uses live LLM calls via a multi-provider fallback chain (Gemini → Groq → OpenRouter) to answer financial questions in context.
- Children interact with pre-loaded story quizzes, gamified financial learning modules, and personality-based feedback.
- Parents benefit from rule-based behavioral analysis dashboards, statistical reports, PDF exports, weekly personality evolution updates, and an AI chatbot coach.

2. Overall Description

2.1 Product Perspective

Money Mirror is a standalone mobile application integrating:

- React Native frontend (cross-platform, Android & iOS)
- ASP.NET Core 8.0 Web API backend with JWT authentication and role-based access
- SQL Server database for all user data, expenses, goals, personality profiles, achievements, and quiz templates
- Python Flask microservice for rule-based personality scoring, behavioral analysis, weekly personality updates, and chatbot routing
- Offline LLM batch pipeline (Claude 3.5 Sonnet) for pre-generating story quiz content only
- Hangfire background scheduler for allowance crediting, age updates, goal expiry, weekly personality updates, and daily reminders
- SignalR for real-time in-app notifications
- Brevo for transactional parent email notifications

2.2 Product Functions

Function 1	**Parent and Child Signup**
Input	Parent email, password, child name, generated login code
Output	Account records created, child login codes generated, confirmation email sent
Processing	Validate inputs, create parent account, generate unique 6-character child login code, store in SQL Server, send email confirmation via Brevo

Function 2	**Multi-Child Support**
Input	Parent login credentials, request to view/manage child profiles
Output	List of children under parent account, selected child profile and data
Processing	Retrieve child profiles linked to parent via ParentChild junction table, enable switching between profiles, fetch associated expenses and goals

Function 3	**Personality Profiling**
Input	Questionnaire responses, ongoing spending logs, mood data, quiz answers
Output	Financial personality profile stored in database with parent-facing and child-facing labels and actionable recommendations
Processing	Rule-based engine (profile_child) scores questionnaire answers. Weekly update engine (weekly_personality_update) evolves scores using behavioral data and quiz results. Four personality types: Impulsive Spender / Speedy Spender, Prudent Saver / Treasure Keeper, Goal-Oriented Planner / Dream Builder, Bargain Hunter / Deal Detective.
Note	*Personality scoring is currently fully rule-based and deterministic. A transition to trained ML models is planned once sufficient user data is collected.*

Function 4	**Allowance Management**
Input	Weekly/monthly allowance amount, bonus settings, child selection
Output	Updated allowance records, Hangfire-scheduled automatic crediting, balance updates, in-app notifications
Processing	Store allowance schedules in SQL Server. Hangfire background job (every 15 minutes) checks and credits due allowances. Bonus payments credited immediately.

Function 5	**Savings Goals Tracking**
Input	Savings goal description, target amount, optional end date, user type (parent/child)
Output	Visual progress tracking, automatic reward crediting on completion, notifications, achievement badges
Processing	Child creates personal goals (max 2 active). Parent creates challenge goals with optional monetary rewards. System auto-fails expired goals and refunds saved amounts. Hangfire job runs daily.

Function 6	**Expense Logging**
Input	Expense amount, category selection, mood emoji (Happy/Sad/Neutral/Excited/Regretful), optional notes
Output	Expense record created, balance deducted, transaction log created, parent notified in real time
Processing	Validate balance sufficiency, create Expense record, deduct from child CurrentBalance, create Transaction record, trigger SignalR notification to parents, check achievement milestones

Function 7	**Expense Viewing**
Input	User credentials, date/category/mood filters
Output	Filtered expense list with amounts, categories, moods, and dates
Processing	Children see kid-friendly view with totals and mood indicators. Parents see detailed list with filter support (date range, category, mood). Both views served from SQL Server.

Function 8	**Reports**
Input	Expense records, mood logs, time period filters, selected sections
Output	Category breakdowns, mood-spending correlations, time pattern analysis, balance history, goal report, exportable PDF
Processing	C# backend aggregates expense data. Report sections: Spending Summary, Category Breakdown, Mood Spending, Time Patterns, Balance History, Goal Report. PDF generation via QuestPDF library.

Function 9	**Interactive Story Quiz**
Input	Child login credentials, pre-stored quiz selection, multiple-choice responses
Output	Story scenario served from database, immediate rule-based feedback, personality score update, badge progress
Processing	Load quiz from StoryQuizTemplates. Evaluate answer using PersonalityType mapping. Update child QuizCount and personality scores. Trigger achievement check.
Note	Quizzes are generated offline in batches using Claude 3.5 Sonnet, validated (exactly 4 choices, all 4 personality types, no duplicates), and stored in the database. No LLM is called during gameplay.

Function 10	**Quiz Batch Generation System (Offline)**
Input	Topic category batches, LLM prompt templates
Output	500 validated quiz questions stored in SQL Server
Processing	LLM generates 10 batches x 50 questions per run. Each response is validated: exactly 4 answer choices, all 4 personality types present exactly once, no duplicate texts. Only valid questions are inserted into the database.

Function 11	**AI Chatbot (Children & Parents)**
Input	User message, conversation history, child profile context (balance, personality, goals, spending data)
Output	Contextual financial advice response tailored to the user's role and child's data
Processing	Child chatbot: filters unsafe content, redirects off-topic messages, passes enriched context to LLM. Parent chatbot: passes child analysis data, alerts, and strengths to LLM. Multi-provider fallback: Gemini → Groq → OpenRouter.

Function 12	**Weekly Personality Evolution**
Input	Weekly behavior logs (spending frequency, savings ratio, mood indicators, goal progress), quiz results, current personality scores
Output	Updated personality score profile, evolved classification, parent notification
Processing	Hangfire job runs every Saturday at 4 AM. Aggregates 30-day behavioral window. Combines behavior scores and quiz answers with recency weighting. Normalizes across 4 personality types. Updates child scores in DB and notifies all linked parents.

Function 13	**Behavioral Analysis Dashboard**
Input	Parent credentials, child ID
Output	Personality profile section, behavioral dimension scores, triggered alert cards, strength cards
Processing	C# AnalysisService evaluates 14-day expense window against 10 threshold-based triggers (e.g., HIGH_MOOD_SPENDING). Returns categorized alert and strength advice cards from pre-seeded AnalysisAdviceTemplate table.

Function 14	**Notification Alerts**
Input	Trigger events (goal progress, expense logged, achievement unlocked, low balance, personality update)
Output	Real-time in-app notifications (SignalR) and email notifications (Brevo) with deep links
Processing	NotificationService sends in-app alerts via SignalR hub groups (parent-{id}, child-{id}). Email alerts sent via Brevo for personality profile updates and weekly reports. Daily expense reminders sent at 4 PM via Hangfire.

Function 15	**Achievements**
Input	Child action data, predefined milestone thresholds
Output	Unlocked badge icons, trophy case display update, in-app notification
Processing	AchievementService checks ExpenseCount, GoalCount, and QuizCount against 12 predefined thresholds across 3 categories. Badges unlocked on threshold crossing and stored in ChildAchievements table.

Function 16	**Character Guide**
Input	Child's selected character (Nova, Cosmo, Luna, Stella), screen context
Output	Character image and contextual message appropriate to current screen
Processing	CharacterService retrieves CharacterState record matching screen context. Five space-themed characters with screen-specific images and messages stored in Cloudinary, referenced in SQL Server.

2.3 User Classes and Characteristics

Money Mirror defines two main user classes to manage access and functionality appropriately:

1. **Parent**
    - Primary account holders. Manage children's accounts, set allowances and challenge goals, view behavioral analysis dashboards, access reports, and receive notifications. Require a detailed, data-rich interface.
2. **Child**
    - Users aged 6-14. Log expenses with mood tracking, participate in story quizzes, track personal savings goals, earn badges, and interact with a character guide. Require a colorful, simple, and encouraging interface.

2.4 Assumptions and Dependencies

1. Reliable internet connection required for chatbot (live LLM calls) and real-time SignalR notifications.
2. All third-party services (Brevo, Gemini, Groq, OpenRouter) are assumed operational.
3. SQL Server and ASP.NET Core are available without additional licensing issues in the academic environment.
4. Offline quiz generation pipeline assumes access to Claude 3.5 Sonnet API during batch runs.

2.5 Design and Implementation Constraints

- **Platform:** React Native for cross-platform Android and iOS. Web/desktop deferred.
- **Hardware:** Minimum 4 GB RAM for smooth rendering and background processing.
- **AI Runtime:** No LLM calls during gameplay or live user interactions (quiz serving, personality display). LLM restricted to offline batch quiz generation and chatbot only.
- **Personality Analysis:** Rule-based system at current stage. ML model integration planned as a future enhancement once training data is sufficient.
- **Backend Architecture:** ASP.NET Core Web API with clean service/interface pattern. Python Flask microservice for AI logic integration only.
- **Scalability:** Multi-child support fully implemented. System designed for incremental quiz content expansion.

2.6 User Documentation

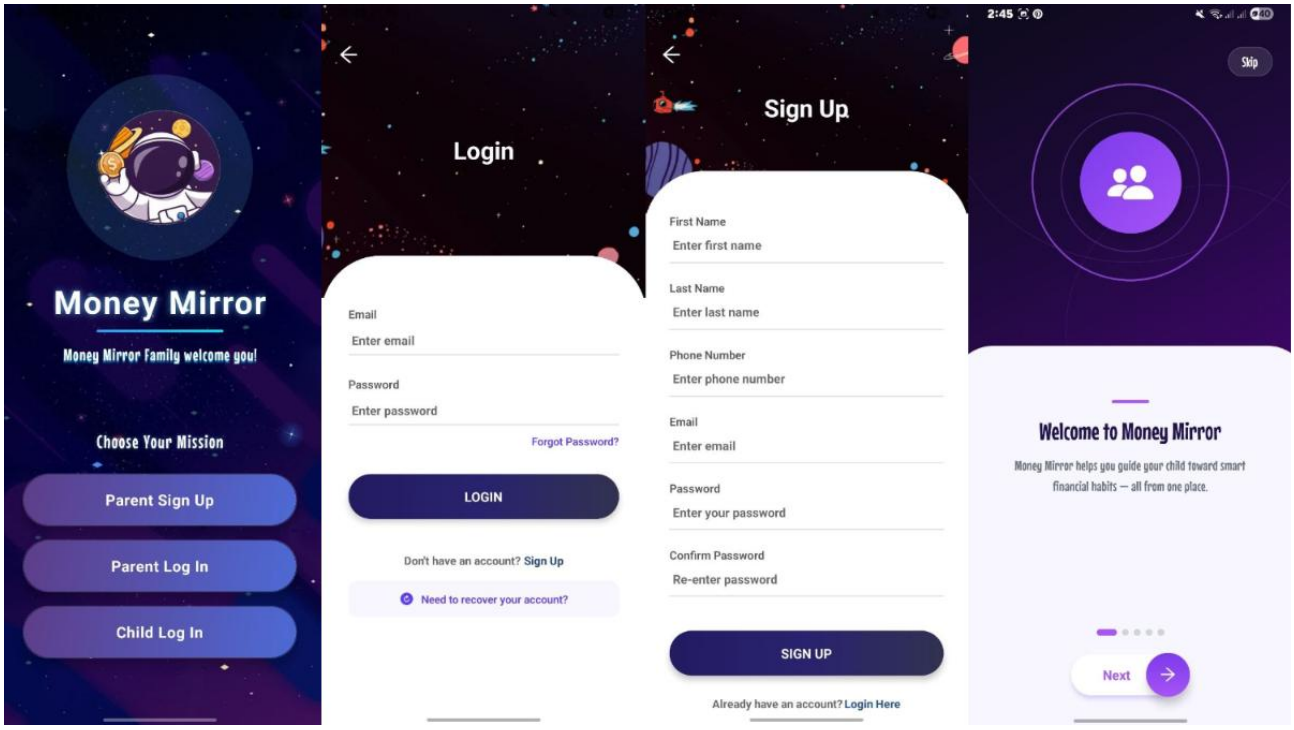
A 5-10 minute video tutorial hosted on YouTube covering: How to Log Expenses, Setting Up Savings Goals, Understanding Personality Profiles, and Using the AI Chatbot. Linked directly from the app's help section.

3. External Interface Requirements

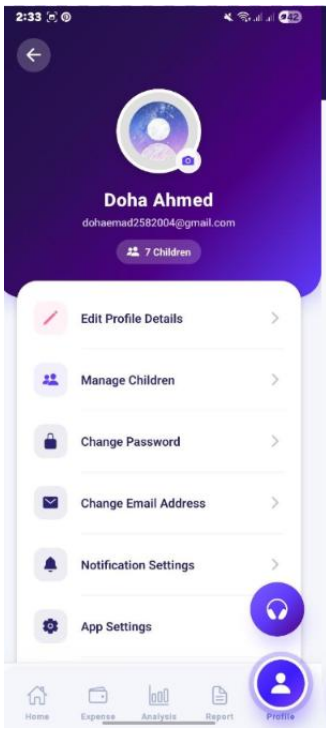
3.1 User Interface Design

The application provides two distinct interface experiences tailored to each user class:

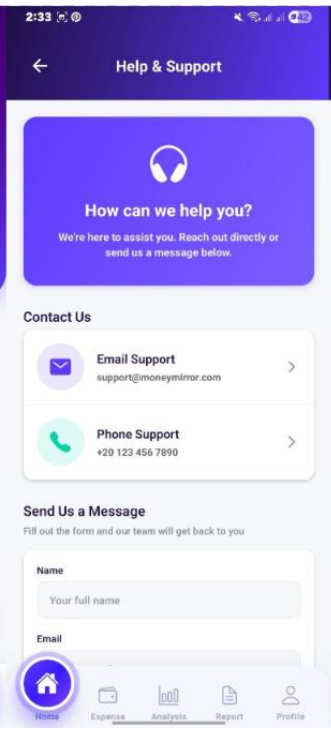
3.1.1 Parent Interface



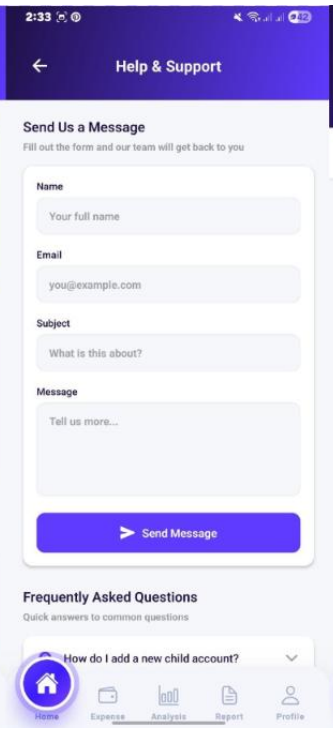
Authentication Page **Parent Login** **Parent Sign Up** **Onboarding**



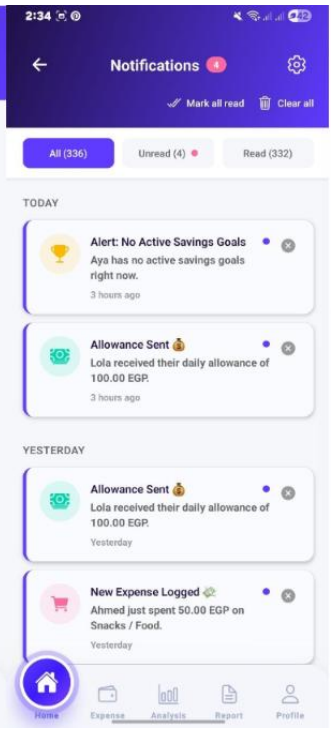
Parent Profile



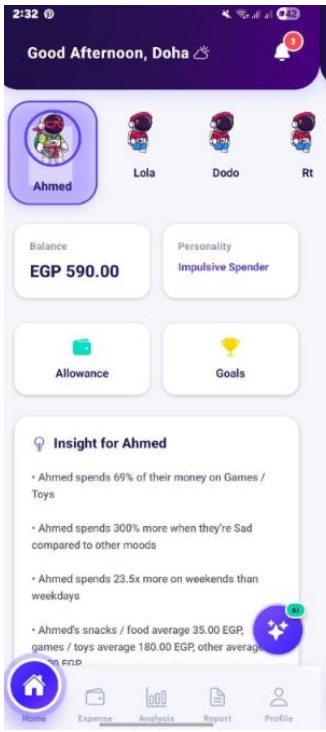
Help and Support 1



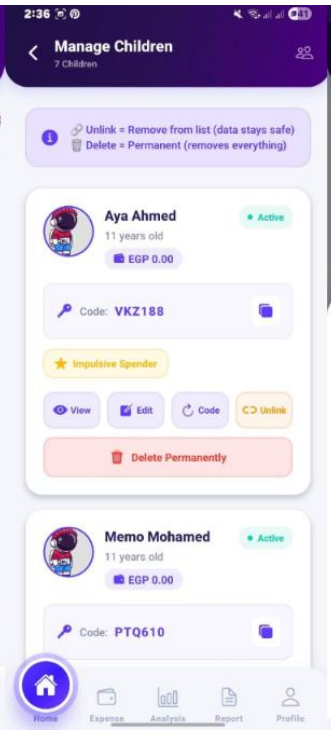
Help and Support 2



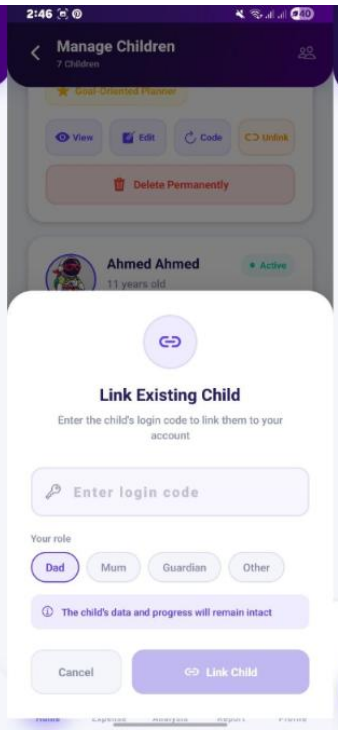
Notifications



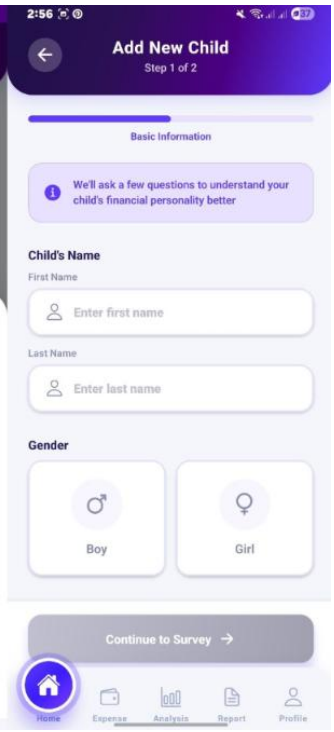
Parent Dashboard



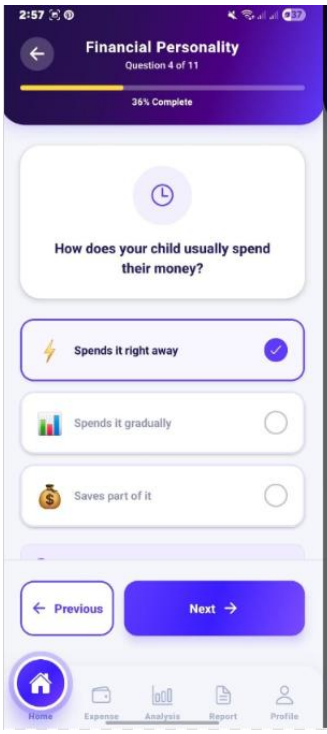
Manage Children



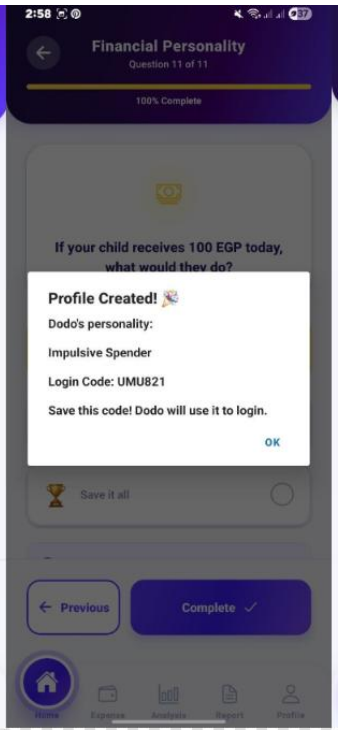
Add Existing Child



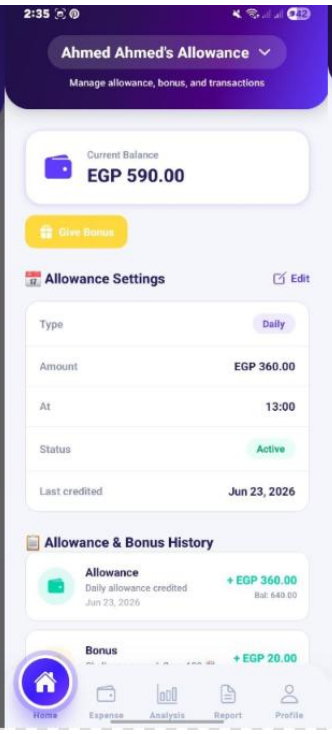
Add New Child



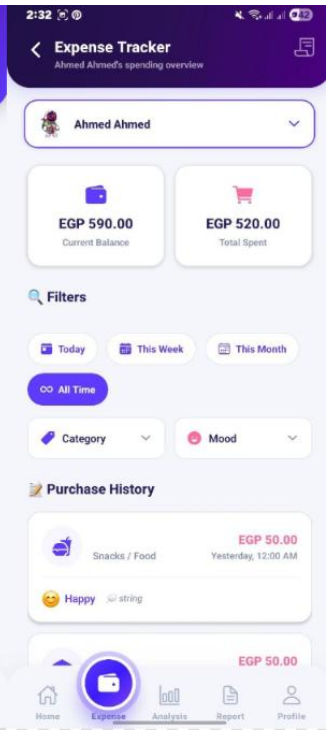
Questionnaire



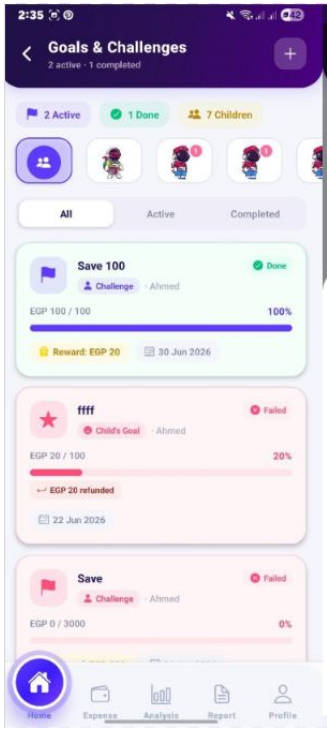
Initial Profiling



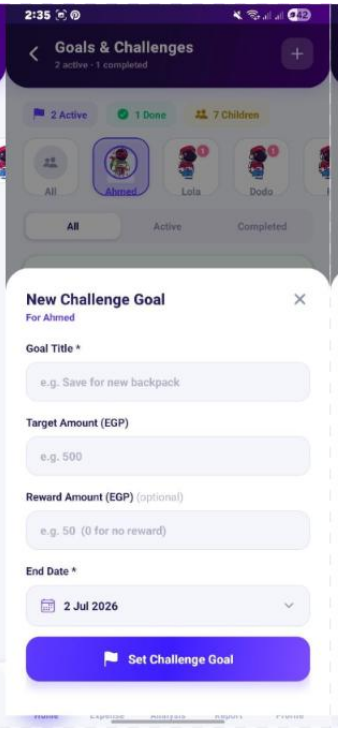
Allowance Management



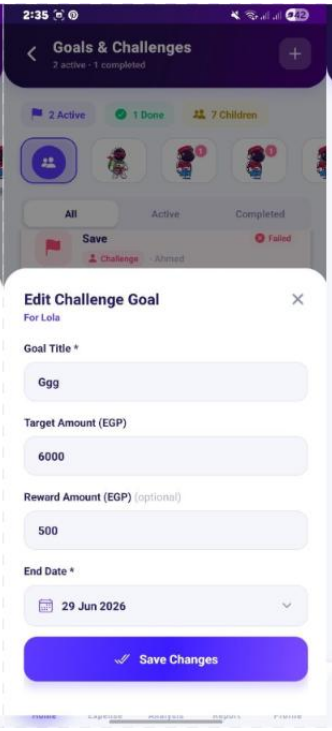
Expenses Tracking



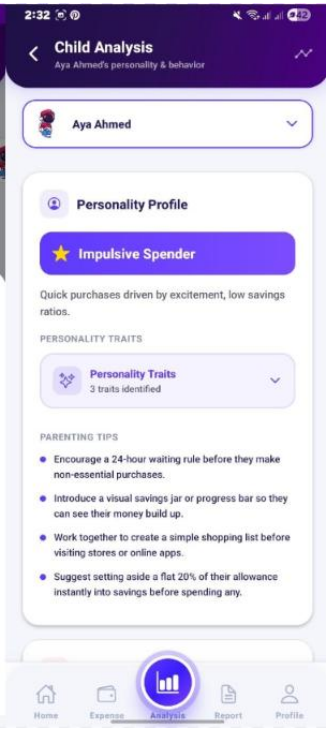
Track Goals & Challenges



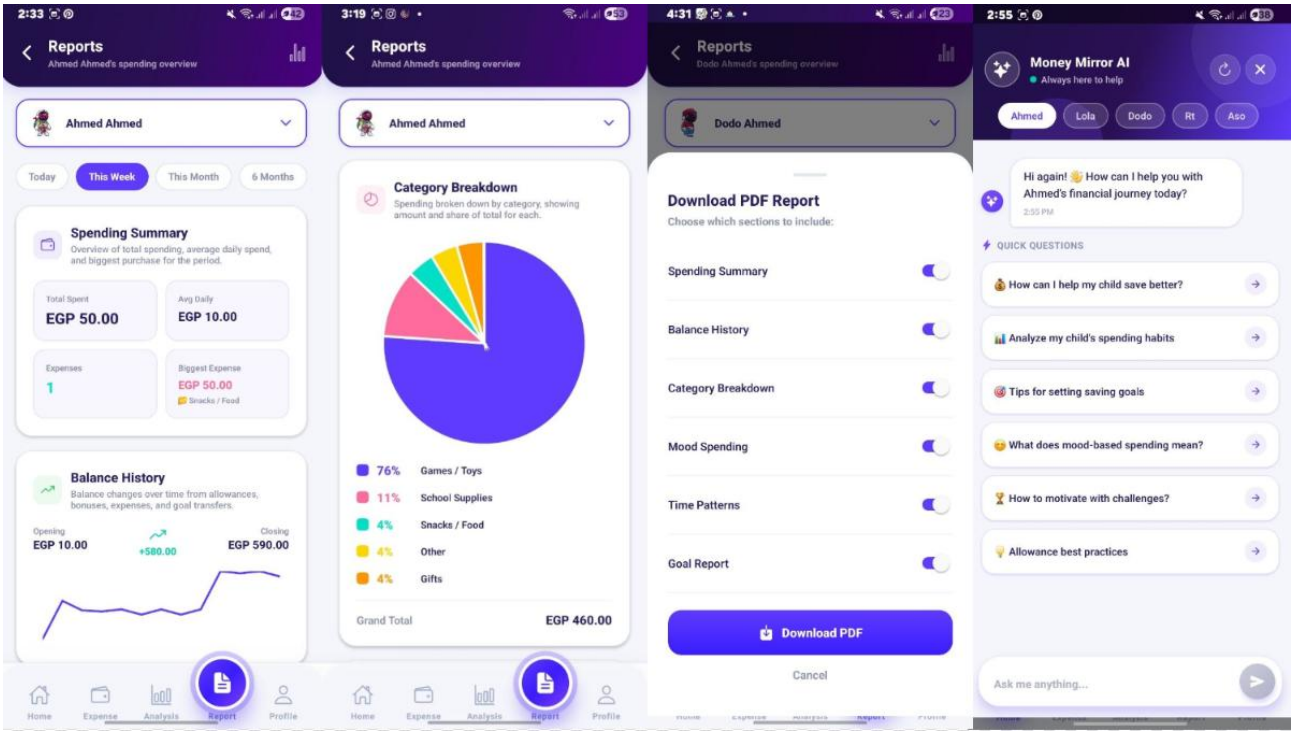
Create Challenge



Edit Challenge Details

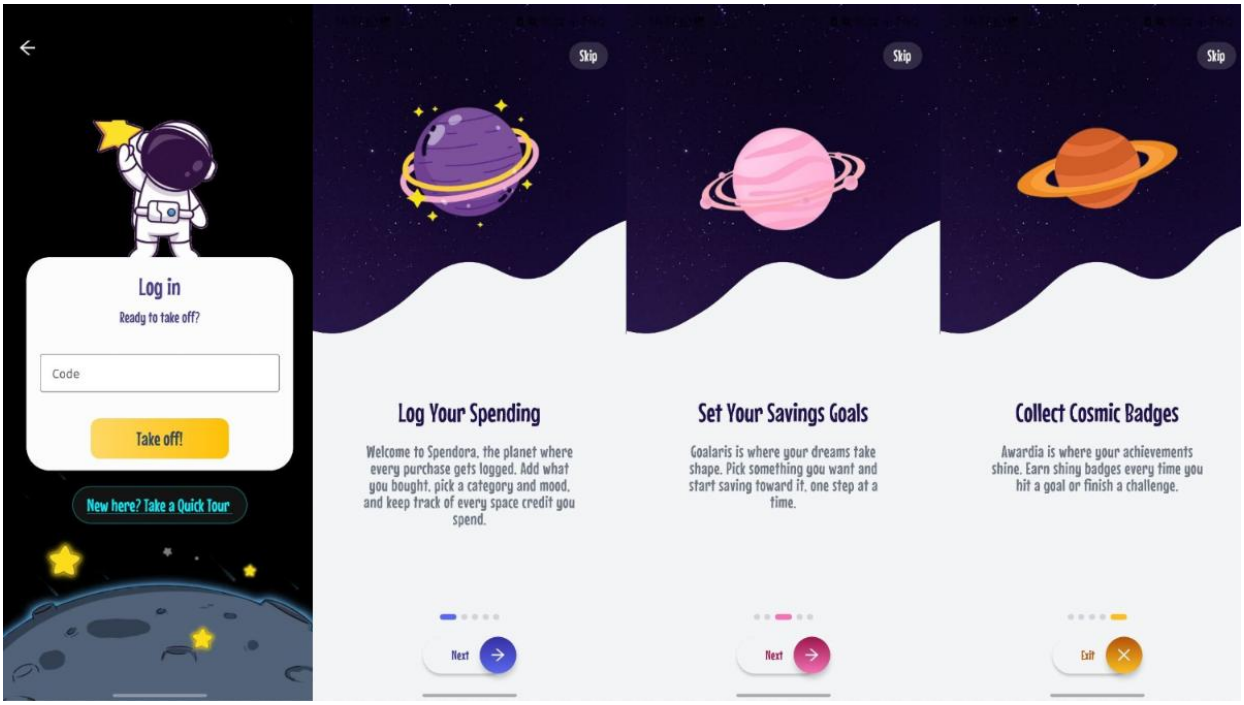


View Behavioural Analysis

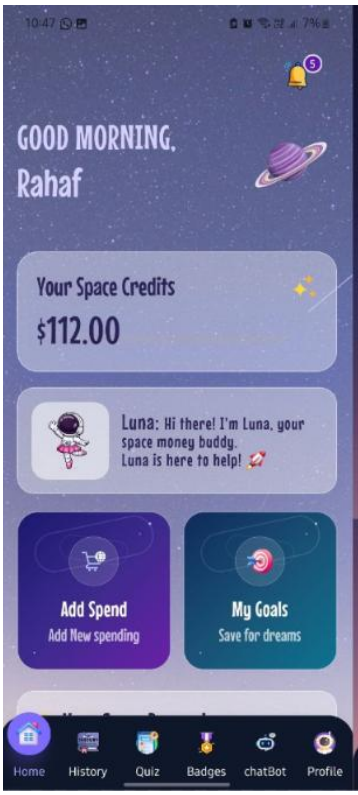


Child Report 1 **Child Report 2** **Download PDF Report** **Chatbot**

3.1.2 Child Interface

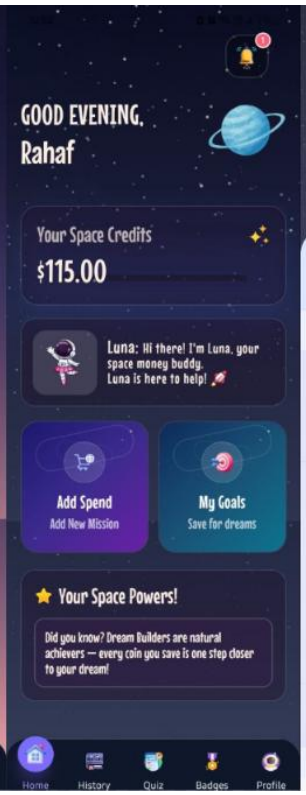


Child Login **Onboarding** **Onboarding** **Onboarding**

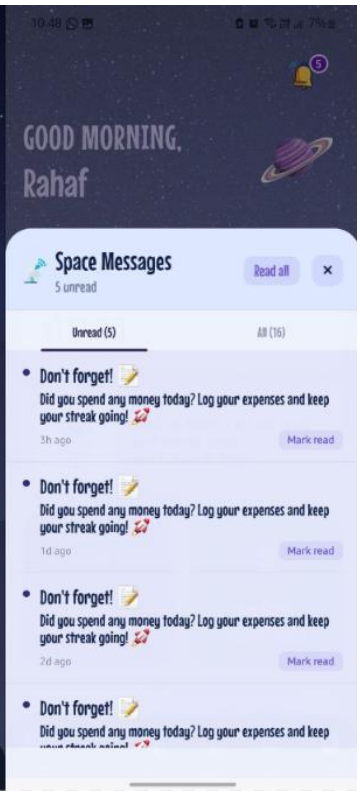


Dashboard (Morning)

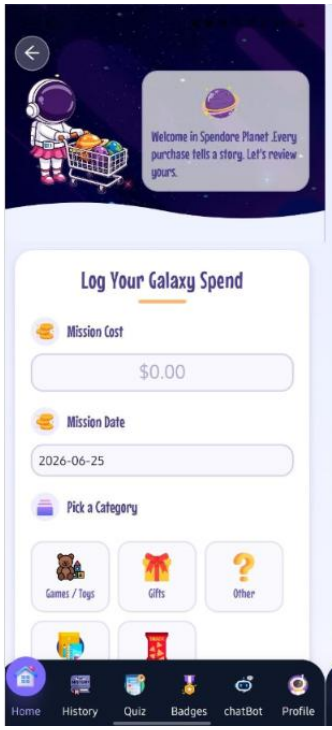
Dashboard (Afternoon)



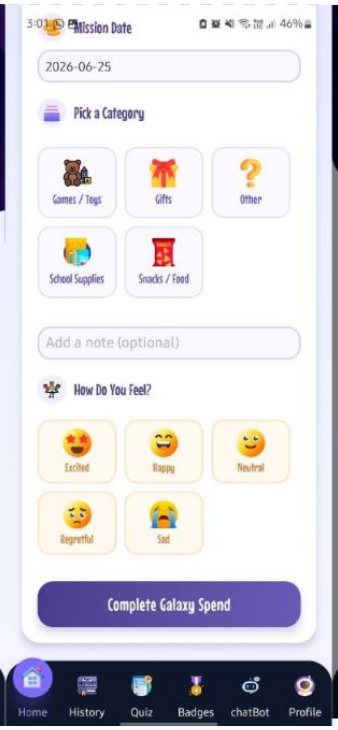
Dashboard (Evening)



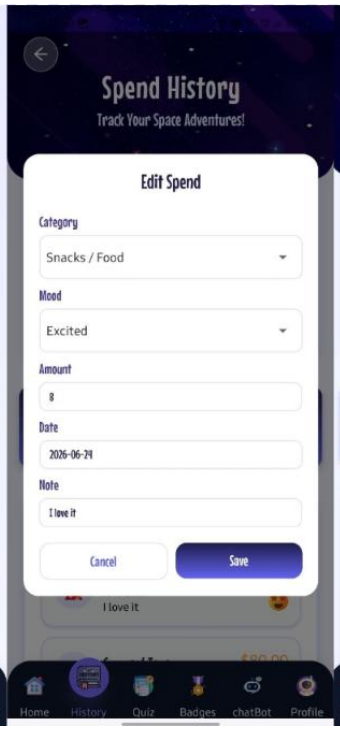
Notifications



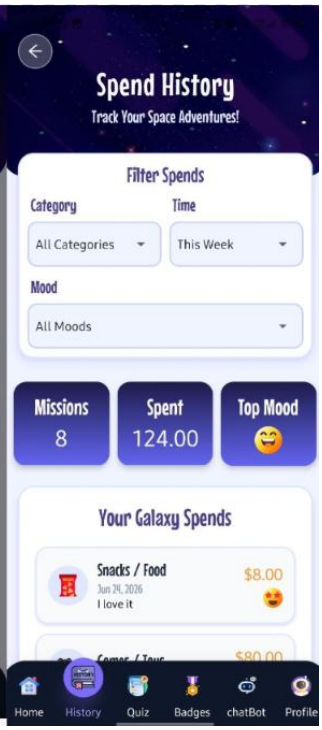
Expense Logging 1



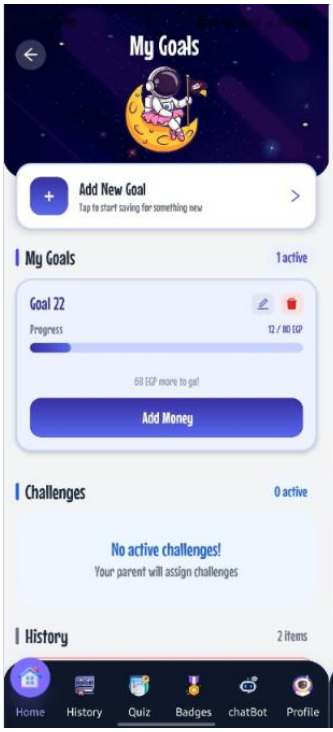
Expense Logging 2



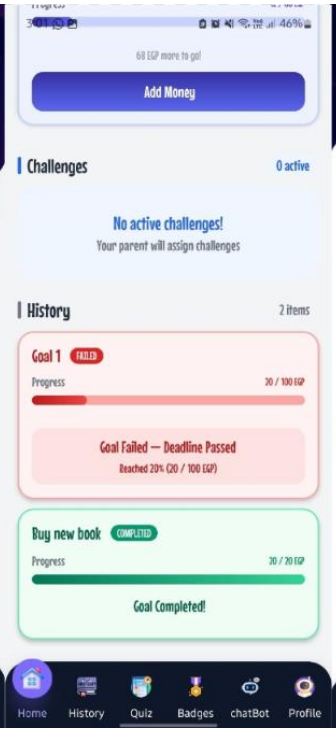
Edit Expense



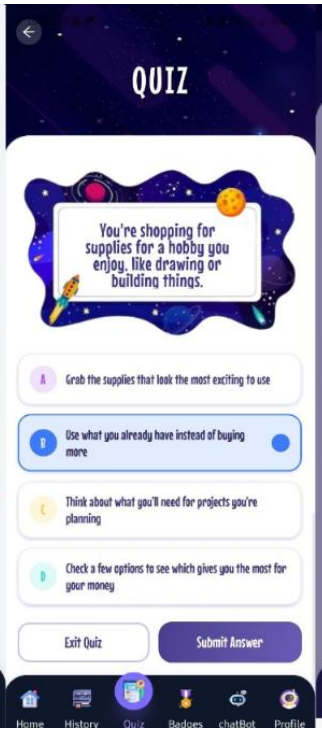
Expense History



Goals and Challenges 1



Goals and Challenges 2

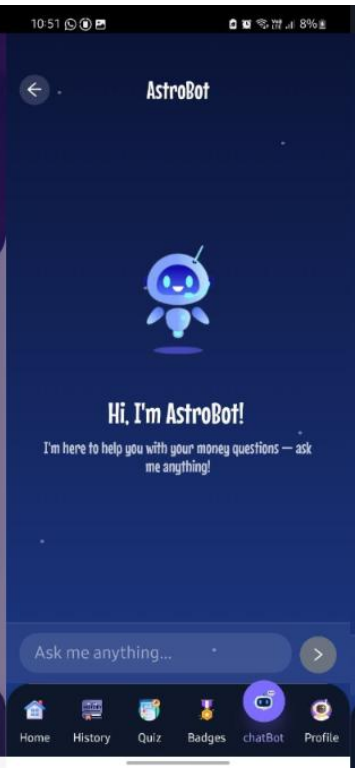


Quiz

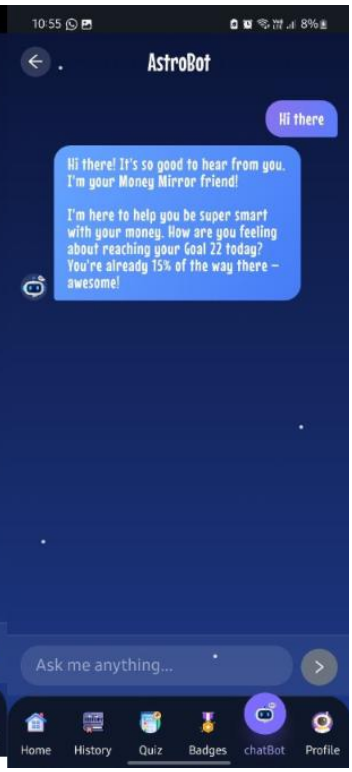
Quiz Feedback

Child Profile

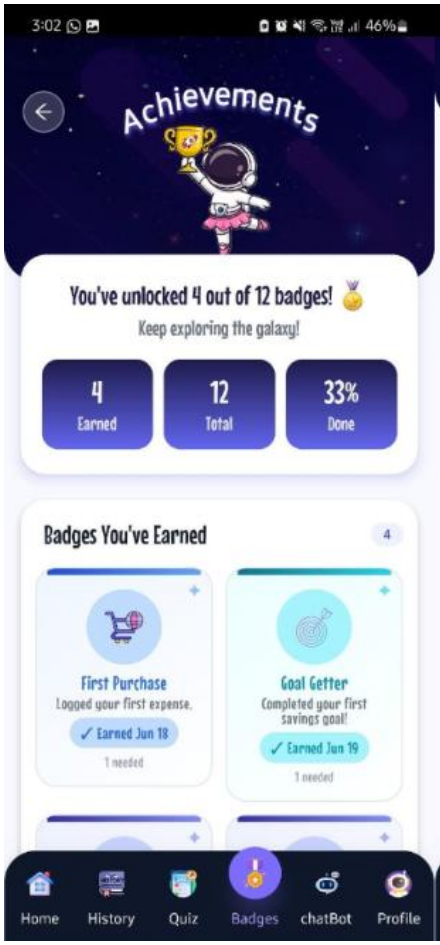
Selecting Avatar/Character



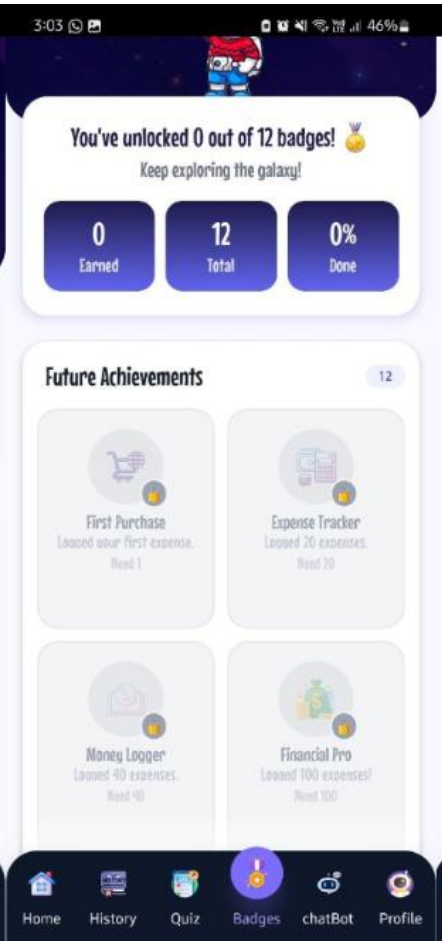
Chatbot 1



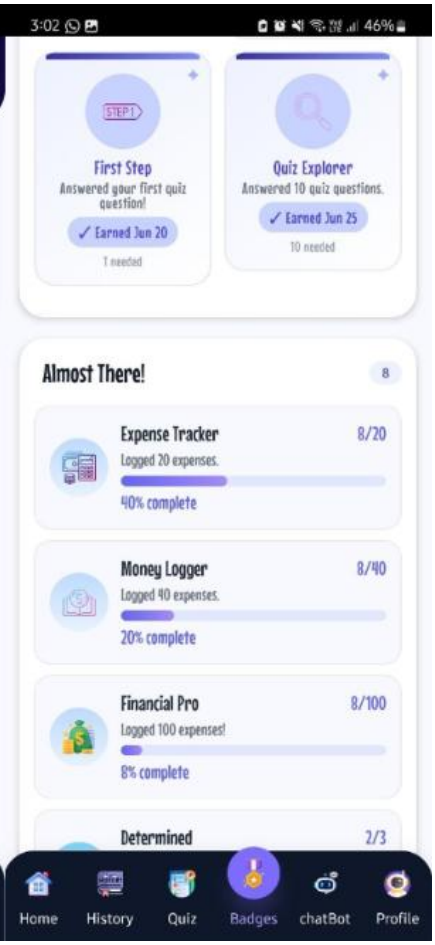
Chatbot 2



Earned Badges



Future Badges



Badges Progress

3.2 Hardware Interfaces

- Touchscreen required for all interactions including swipe navigation and emoji selection
- Minimum 1.5 GHz processor and 4 GB RAM
- Supported: Android 9.0+ and iOS 11+ smartphones/tablets
- Network: Wi-Fi or mobile data for all sync and chatbot features

3.3 Software Interfaces

- Frontend: React Native (cross-platform mobile UI)
- Backend API: ASP.NET Core Web API 8, all business logic, authentication, data access.
- Database: SQL Server 2021, all persistent data storage.
- AI/Python Layer: Python Flask microservice — personality scoring, behavior analysis, chatbot routing, weekly update logic
- LLM (Batch Only): Claude 3.5 Sonnet via OpenAI-compatible API — offline quiz generation only
- Chatbot LLM: Gemini 2.5 Flash (primary), Groq LLaMA-3.3-70B (fallback 1), OpenRouter DeepSeek (fallback 2)
- Background jobs: Hangfire with SQL Server storage — allowance crediting (15-min), daily reminders, goal expiry, age updates, weekly personality update, account cleanup
- Real-time Notifications: ASP.NET Core SignalR — in-app push notifications to grouped hub connections
- Email: Brevo (formerly Sendinblue) — transactional emails for parents (confirmation, password reset, personality updates, weekly reports)
- PDF Generation: QuestPDF — server-side PDF report generation
- Authentication: JWT Bearer tokens (15-min access token, 7-day refresh token). Role-based: Parent vs Child

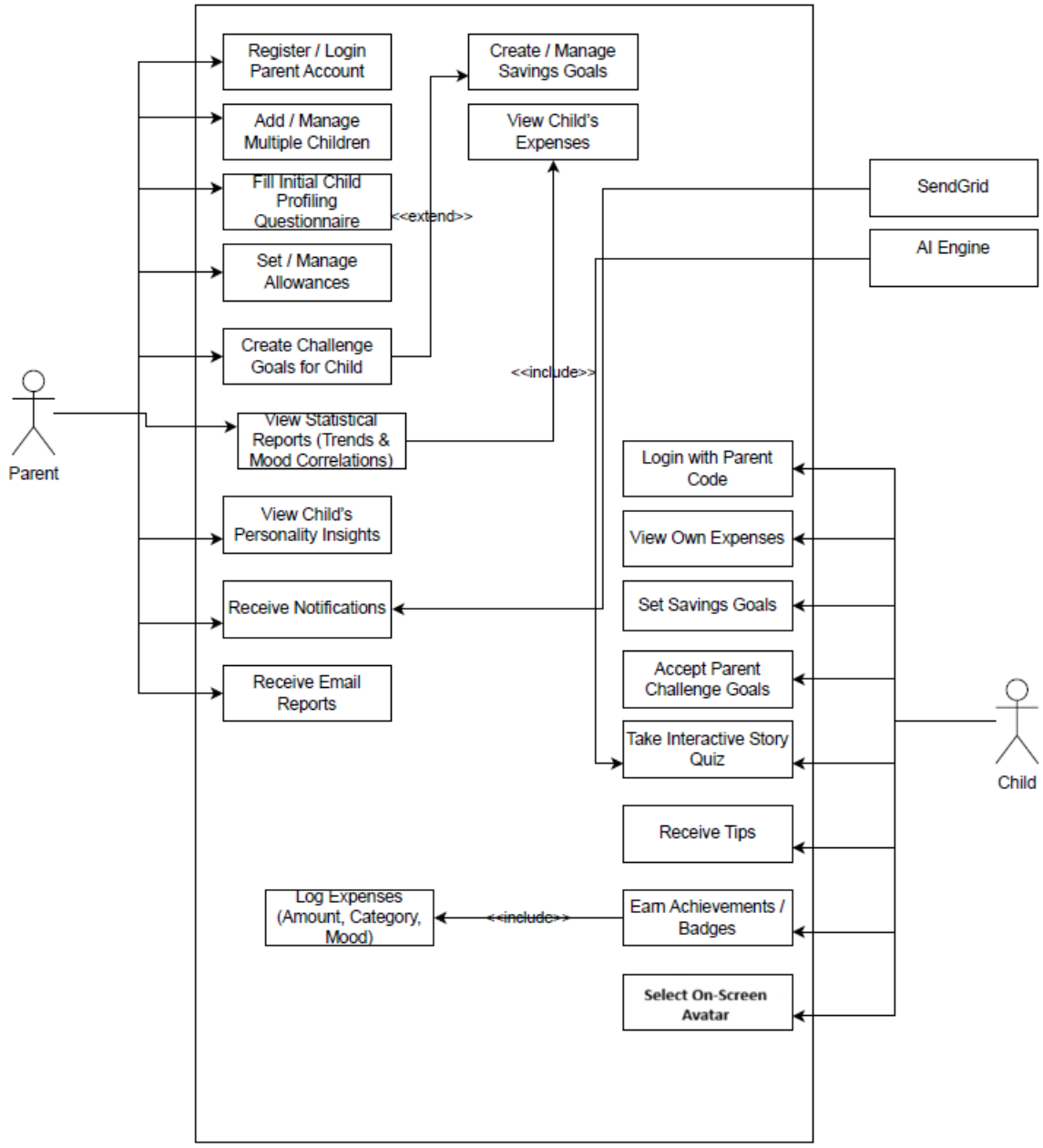
3.4 Communications Interfaces

- HTTPS for all API communication between frontend and backend
- REST API between ASP.NET Core backend and Python Flask microservice
- WebSocket (SignalR) for real-time notification delivery
- SMTP via Brevo for parent email notifications

4. Specific Requirements

4.1 Database Design

4.2 Use Case Diagram



4.3 Detailed Use Cases

Use Case	**Parent Signup | ID: UC-1a | Priority: High**
Actor	Parent
Description	Allows a parent to create an account using email and password.
Trigger	A parent wants to create an account.
Preconditions	App is available. Parent has a valid email address.
Normal Course	1. Parent selects Sign Up. 2. Enters email and password. 3. System validates and creates account. 4. System sends 6-digit email confirmation code via Brevo. 5. Parent enters code to confirm email. 6. Parent is automatically logged in.
Post Conditions	Parent account stored in DB. Email confirmed. JWT tokens issued.

Use Case	**Child Login Using Parent Code | ID: UC-1b | Priority: High**
Actor	Child
Description	Allows a child to log in using the unique 6-character alphanumeric code provided by the parent.
Trigger	A child wants to log into the app.
Preconditions	App is available. Child's login code exists in the system.
Normal Course	1. Child selects Child Login. 2. Enters the 6-character code. 3. System verifies code and loads child profile. 4. JWT tokens issued. 5. Child sees their dashboard.
Post Conditions	Child is authenticated. Session linked to the correct parent account.

Use Case	**Multi-Child Support | ID: UC-2 | Priority: High**
Actor	Parent
Description	Allows parents to manage multiple children's accounts, switching between them on the dashboard.
Trigger	A parent wants to view or manage a different child's account.
Preconditions	Parent is logged in. Multiple child accounts exist under the parent.
Normal Course	1. Dashboard shows all linked children as cards. 2. Parent selects a child. 3. App loads selected child's data (balance, expenses, goals, personality). 4. Parent can switch children at any time.
Post Conditions	Parent views and manages data for the selected child. Data is securely isolated per child.

Use Case	**Initial Profiling & Child Code Generation | ID: UC-3 | Priority: High**
Actor	Parent
Description	Parent completes a questionnaire when adding a new child to establish a preliminary financial personality profile and generate a unique login code.
Trigger	Parent adds a new child to their account.
Preconditions	Parent is logged in. New child record is being added.
Normal Course	1. Parent selects Add New Child. 2. Enters child's basic info (name, DOB, gender, relationship). 3. Completes the 9-question profiling questionnaire. 4. System calls Python AI service to calculate initial personality type. 5. System generates unique 6-character login code. 6. Parent receives the code to share with the child.
Post Conditions	Child record created. Personality profile assigned. Login code generated. Parent-child link stored.

Use Case	**Allowance Management | ID: UC-4 | Priority: High**
Actor	Parent
Description	Parents set daily/weekly/monthly allowances for each child, give one-time bonuses, and view transaction history.
Trigger	Parent wants to manage a child's allowance.
Preconditions	Parent is logged in. Child account exists.
Normal Course	1. Parent selects a child. 2. Navigates to Allowance section. 3. Sets allowance type (Daily/Weekly/Monthly), amount, and schedule. 4. System stores schedule in DB. 5. Hangfire job automatically credits allowance on schedule. 6. Parent optionally gives one-time bonus with reason. 7. Child receives in-app notification upon credit.
Post Conditions	Allowance schedule stored. Balance updated automatically. Transaction record created.

Use Case	**Child Savings Goals | ID: UC-5a | Priority: High**
Actor	Child
Description	Child creates personal savings goals (max 2 active) and tracks progress with visual bars.
Trigger	Child wants to create or manage a savings goal.
Preconditions	Child is logged in.
Normal Course	1. Child selects Savings Goals. 2. Creates a new goal (name, target amount, optional end date). 3. Child adds money from their balance to the goal. 4. Progress bar updates. 5. On completion, system marks goal as Success and notifies all linked parents.
Post Conditions	Goal stored in DB. Balance reduced. Parents notified. Achievement checked.

Use Case	**Parent Challenge Goals | ID: UC-5b | Priority: High**
Actor	Parent
Description	Parent sets challenge goals for children with optional monetary rewards. Monitors progress via notifications.
Trigger	Parent wants to set a financial challenge for a child.
Preconditions	Parent is logged in. Child account exists.
Normal Course	1. Parent selects a child and navigates to Challenge Goals. 2. Creates challenge (title, target amount, end date, optional reward). 3. Child receives in-app notification of new challenge. 4. Child adds money to the goal. 5. On completion, reward is automatically credited to child's balance. 6. Parent receives completion notification.
Post Conditions	Challenge stored. Child notified. Reward auto-credited on completion.

Use Case	**Expense Logging | ID: UC-6 | Priority: High**
Actor	Child
Description	Child logs a purchase with amount, category, mood, and optional notes. Balance is immediately deducted.
Trigger	Child made a purchase and wants to record it.
Preconditions	Child is logged in. Child has sufficient balance.
Normal Course	1. Child selects Log Expense. 2. Enters amount, category, and mood emoji. 3. Optionally adds item name (required for 'Other' category) and a note. 4. System validates balance. 5. Balance deducted. 6. Expense and Transaction records created. 7. Parents notified in real time via SignalR.
Post Conditions	Expense recorded. Balance updated. Transaction logged. Parents notified. Achievement checked.

Use Case	**Expense Viewing — Child | ID: UC-7a | Priority: High**
Actor	Child
Description	Child views their expense history in a kid-friendly interface with category and time filters.
Trigger	Child wants to review their purchase history.
Preconditions	Child is logged in and has logged expenses.
Normal Course	1. Child selects Expense History. 2. Views list of purchases with name, date, mood, and amount. 3. Can filter by category or date range. 4. App shows total count and total spent.
Post Conditions	Child views purchase data in an engaging, age-appropriate format.

Use Case	**Expense Viewing — Parent | ID: UC-7b | Priority: High**
Actor	Parent
Description	Parent views detailed expense history for a selected child with advanced filters.
Trigger	Parent wants to review a child's spending.
Preconditions	Parent is logged in. Child has logged expenses.
Normal Course	1. Parent selects a child. 2. Opens Expense section. 3. Filters by date, category, or mood. 4. Views list with item, date, mood, and amount. 5. Sees summary: total spent, count, top categories, mood distribution.
Post Conditions	Parent gains detailed insights into child's spending behavior.

Use Case	**Behavioral Analysis Dashboard | ID: UC-8 | Priority: High**
Actor	Parent
Description	Parent views rule-based behavioral analysis of child's spending patterns including personality profile, behavioral dimension scores, and triggered alert/strength cards.
Trigger	Parent requests behavior insights for a child.
Preconditions	Parent is logged in. Child has expense and goal data.
Normal Course	1. Parent selects Analysis for a child. 2. App shows personality profile (type, description, traits, recommendations). 3. If personality scores exist: behavioral dimension chart (Impulsive %, Prudent %, Planner %, Bargain %). 4. Alert cards for detected issues (e.g., high mood-driven spending, low savings). 5. Strength cards for positive behaviors. 6. Data window note (last 14 days).
Post Conditions	Parent views evidence-based behavioral insights and actionable guidance.

Use Case	**Child Personality Profile View | ID: UC-9 | Priority: High**
Actor	Child
Description	Child views their personality type in a fun, age-appropriate format with encouraging tips.
Trigger	Child opens their profile section.
Preconditions	Child is logged in.
Normal Course	1. Child selects Profile. 2. App displays selected character avatar and option to change it. 3. Shows child-friendly personality name (e.g., Speedy Spender, Dream Builder). 4. Displays fun facts and personalized tips. 5. Shows fun facts: favorite category, most common spending mood, biggest purchase this month.
Post Conditions	Child receives engaging, positive feedback reinforcing healthy financial habits.

Use Case	**Statistical Reports | ID: UC-10 | Priority: High**
Actor	Parent
Description	Parent accesses detailed spending reports with charts, breakdowns, and PDF export.
Trigger	Parent requests a statistical report.
Preconditions	Parent is logged in. Child has expense data.
Normal Course	1. Parent opens Reports section. 2. Selects report type and date range. 3. Views: Spending Summary, Category Breakdown, Mood-Spending Correlation, Time Patterns, Balance History, Goal Report. 4. Charts display data visually. 5. Parent optionally downloads selected sections as a PDF.
Post Conditions	Parent views comprehensive spending analytics with PDF export option.

Use Case	**Interactive Story Quiz | ID: UC-11 | Priority: High**
Actor	Child
Description	Child participates in story-based financial quizzes (pre-stored in DB) and receives immediate feedback.
Trigger	Child selects the Story Quiz option.
Preconditions	Child is logged in. Quiz data is available in the database.
Normal Course	1. Child opens Quiz section. 2. App loads next age-appropriate, unplayed quiz from DB (sequential by ID). 3. Child reads story scenario and selects one of 4 choices. 4. System records answer, updates QuizCount and personality scores. 5. Character reacts: happy animation for good financial choice, sad for poor choice. 6. Feedback message shown. 7. Achievement check triggered. Limit: 2 quizzes per day.
Post Conditions	Answer logged. Personality scores updated. Achievement checked. Daily limit enforced.

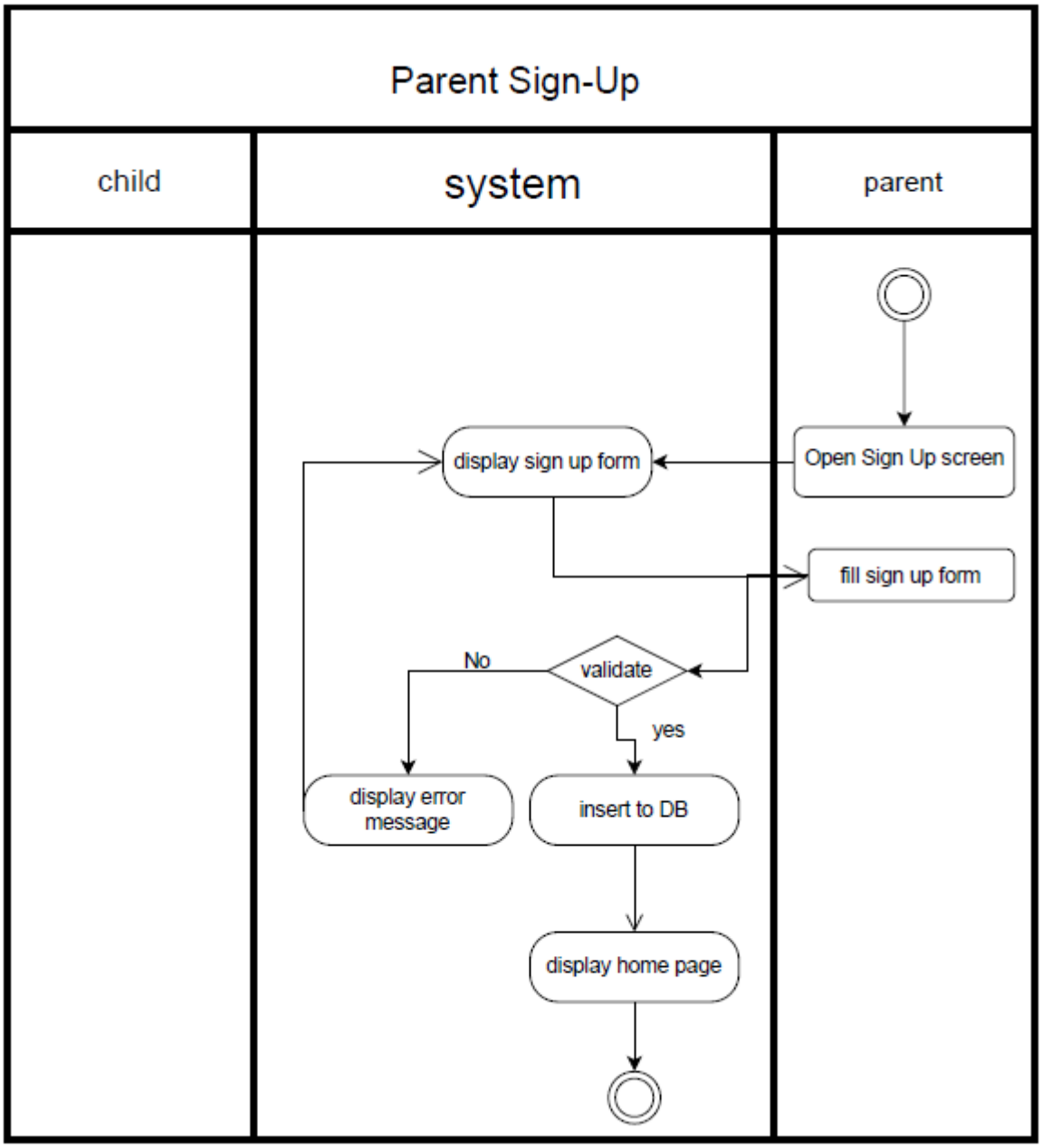
Use Case	**AI Chatbot | ID: UC-12 | Priority: High**
Actor	Parent, Child
Description	Context-aware financial chatbot for both parents and children using live LLM with multi-provider fallback.
Trigger	User opens the chatbot and sends a message.
Preconditions	User is authenticated. Internet connection available.
Normal Course	1. User opens Chatbot. 2. Types a financial question. 3. System enriches with user context (child profile, balance, spending patterns, personality, goals). 4. Child chatbot: filters unsafe content, redirects off-topic queries. Parent chatbot: attaches analysis alerts and strengths. 5. LLM (Gemini first, then Groq, then OpenRouter) generates response. 6. Response displayed to user.
Post Conditions	User receives contextual financial advice or redirection.

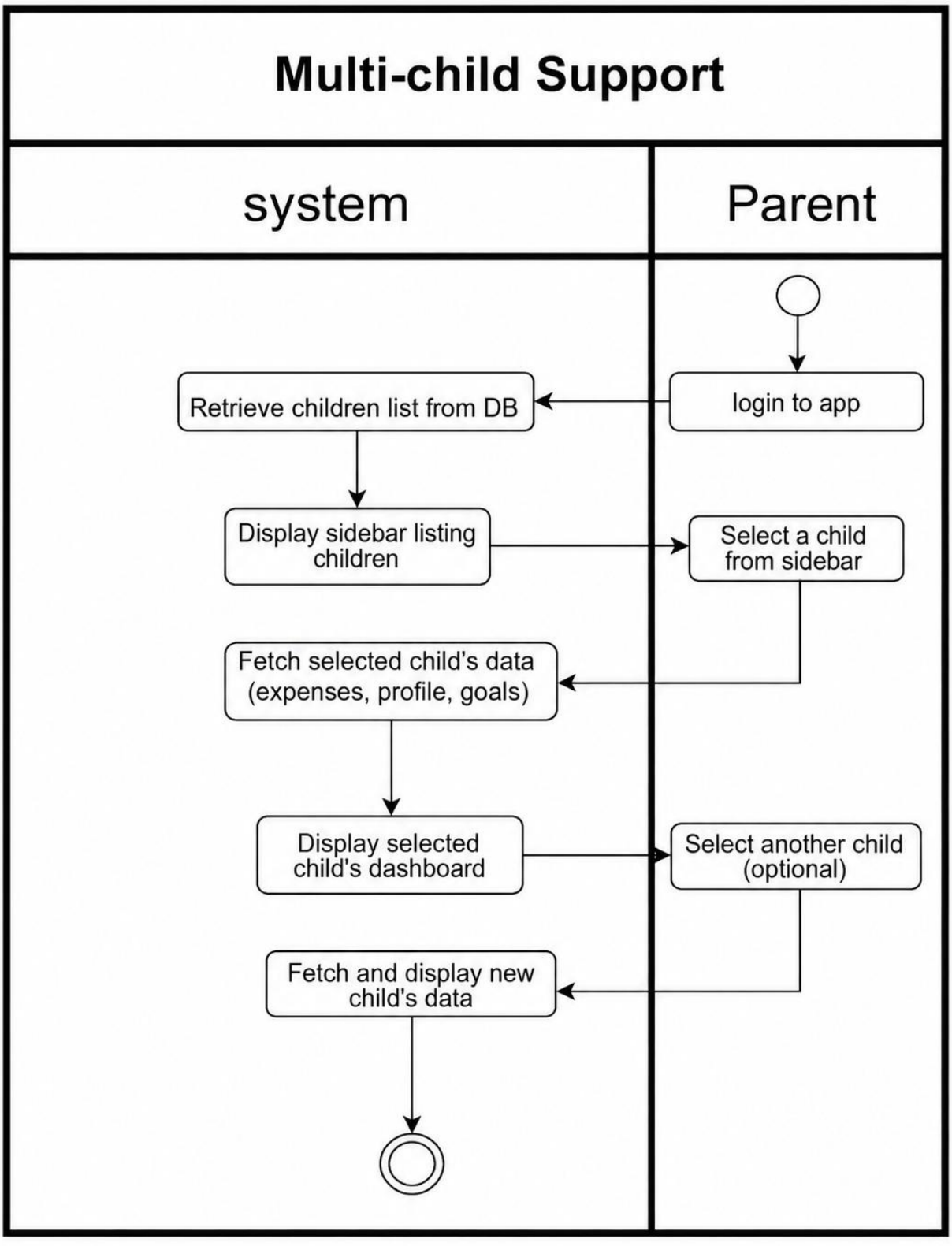
Use Case	**Notification Alerts | ID: UC-13 | Priority: High**
Actor	Parent, Child
Description	Real-time in-app notifications and email alerts triggered by system events.
Trigger	System event occurs (expense logged, goal completed, achievement unlocked, low balance detected, etc.)
Preconditions	User is authenticated.
Normal Course	1- In-App: SignalR sends notification to user's hub group. 2- Notification stored in DB. 3- Unread count badge updated. 4- Email (Parent only): Brevo sends email for personality profile updates, weekly reports, and confirmation flows.
Post Conditions	Notification stored. User informed. Email sent to parent where applicable.

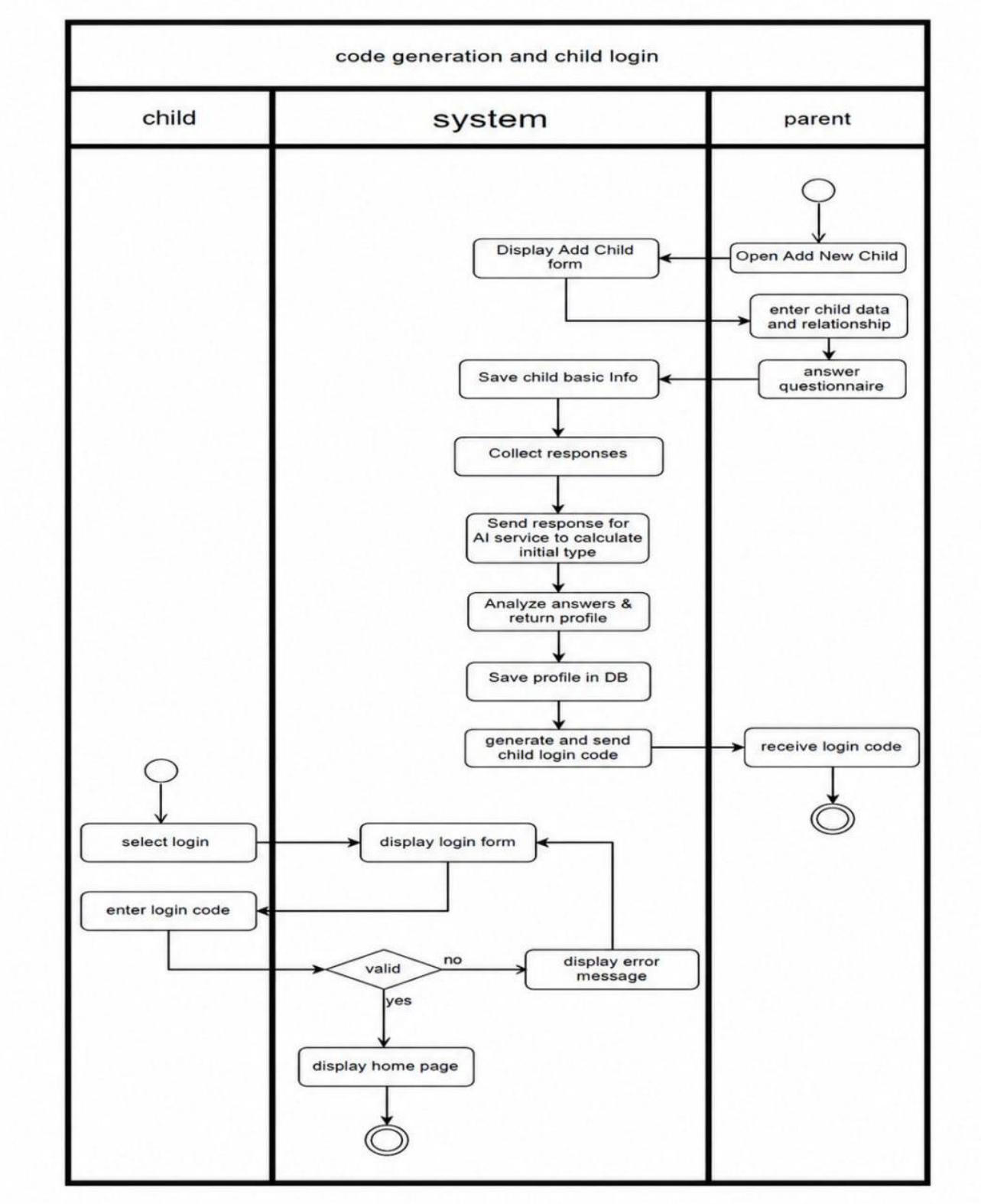
Use Case	**Earning Achievements | ID: UC-14 | Priority: High**
Actor	Child
Description	Children earn badges when they reach milestone thresholds in Quiz answers, Goals completed, or Expenses logged.
Trigger	Child completes an action that may trigger an achievement.
Preconditions	Child is logged in. Achievement logic is active.
Normal Course	1. Child performs qualifying action. 2. AchievementService checks current count against thresholds. 3. If threshold reached and not already earned: badge unlocked. 4. ChildAchievement record created. 5. In-app notification sent. 6. Badge appears in Trophy Case.
Post Conditions	Badge earned and displayed. Notification sent. DB updated.

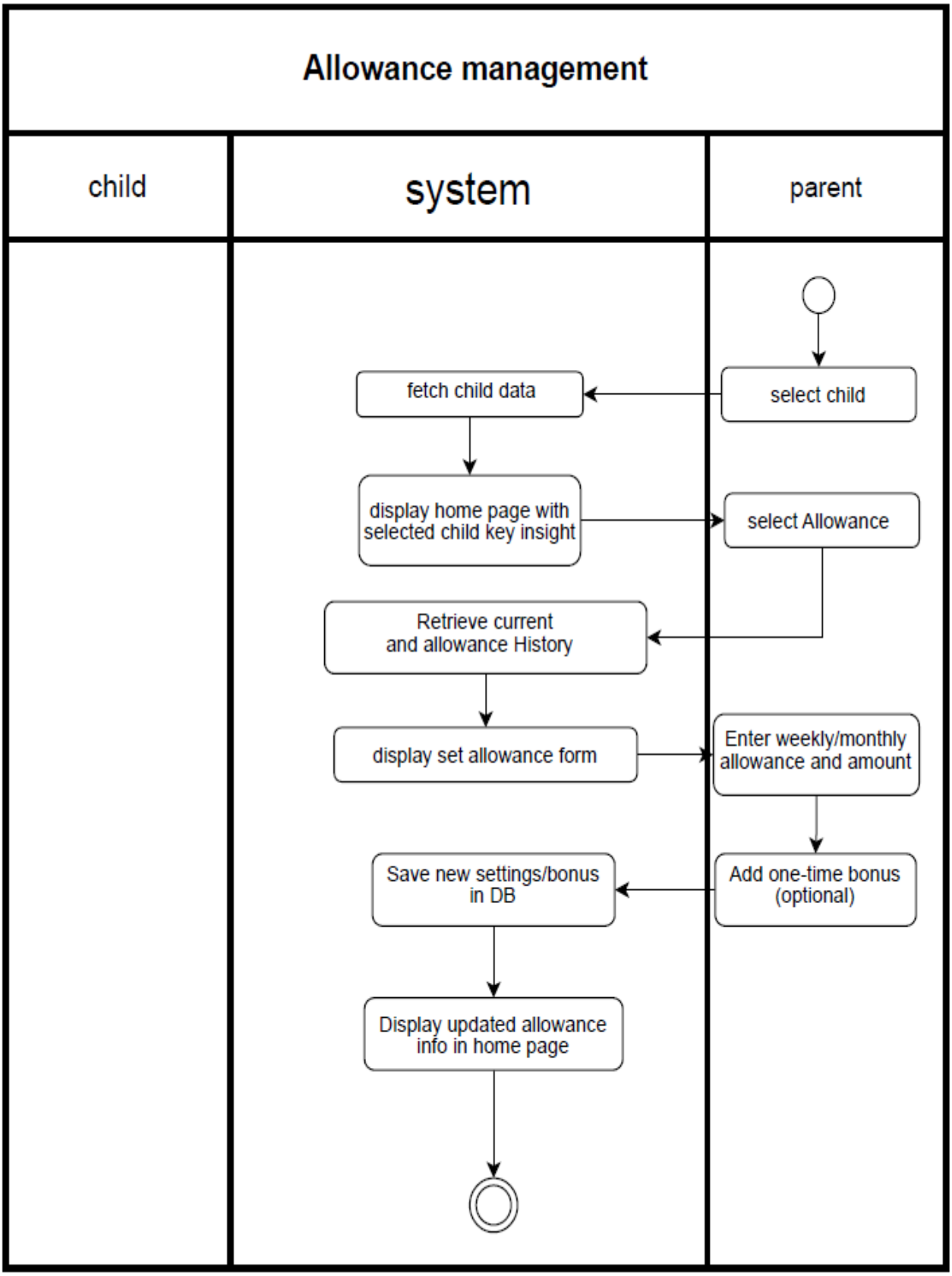
Use Case	**Email Notifications for Parents | ID: UC-15 | Priority: High**
Actor	Parent
Description	Parents receive email notifications for important system events including account confirmation, password reset, personality profile updates, and weekly reports.
Trigger	System detects a triggering event (registration, personality update, report ready, etc.)
Preconditions	Parent has a verified email address.
Normal Course	1. System detects event. 2. EmailService builds HTML email template. 3. Sends via Brevo to parent's registered email. 4. Email contains event summary. 5. Notification log stored in DB.
Post Conditions	Parent receives timely email with relevant details and app link.

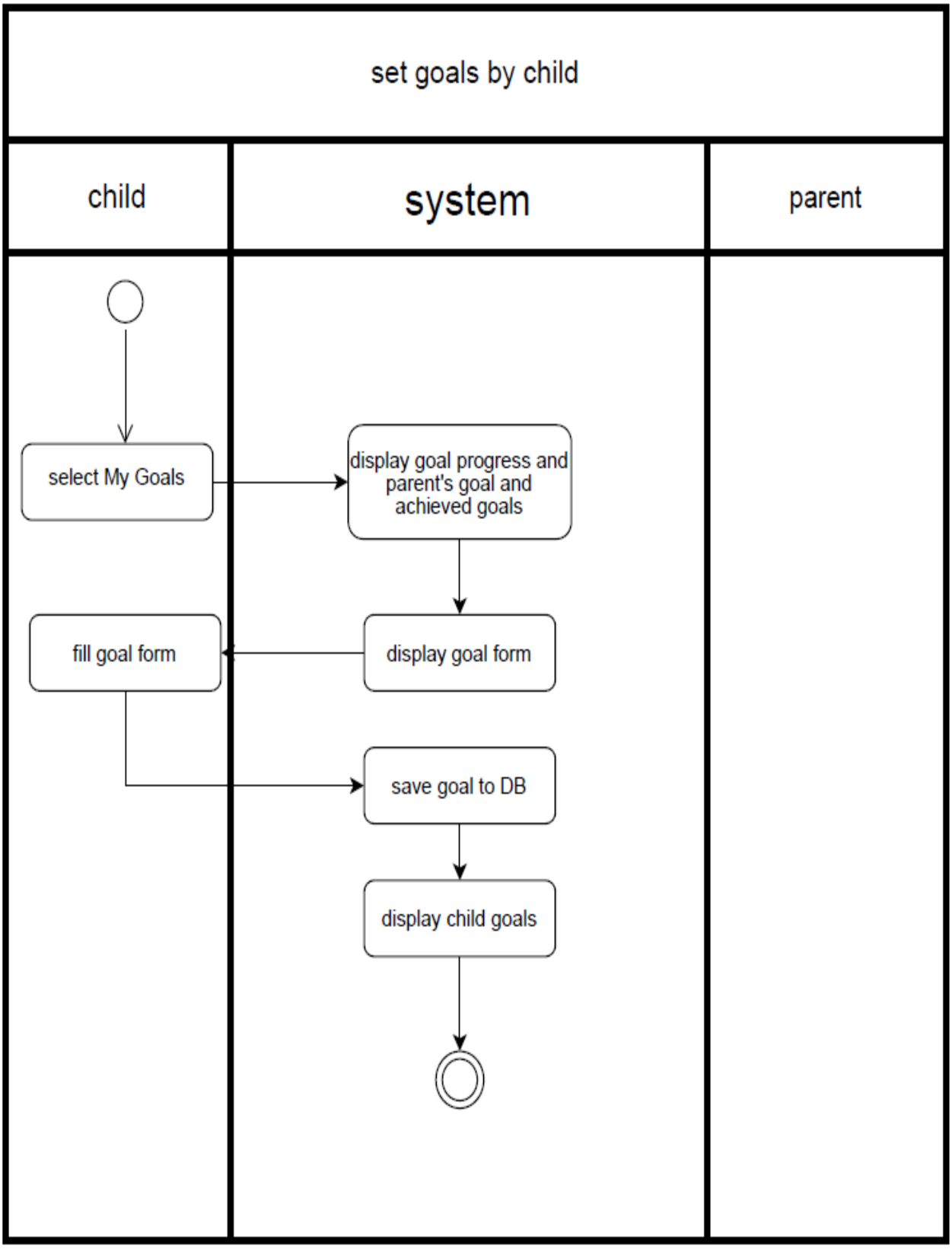
4.4 Activity Diagram

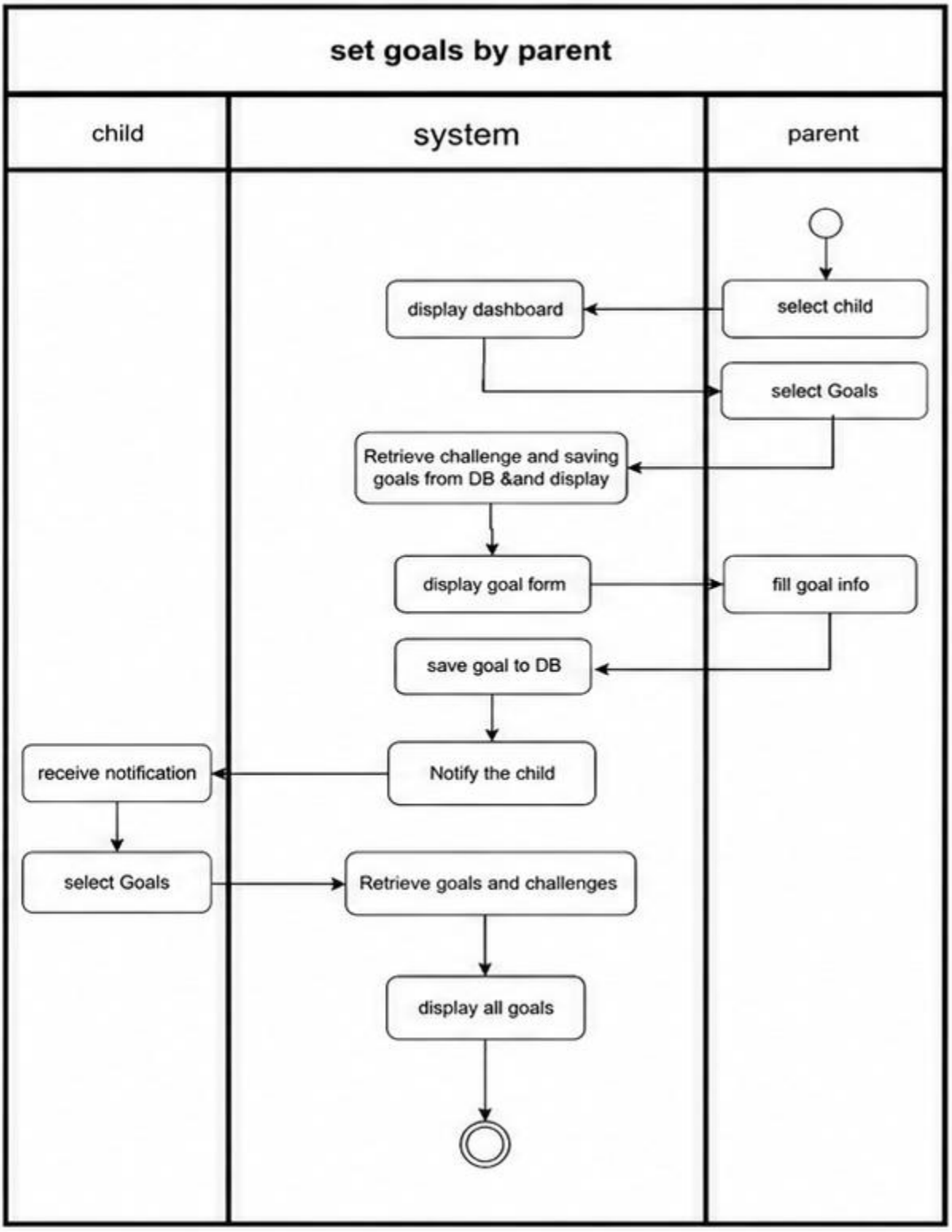


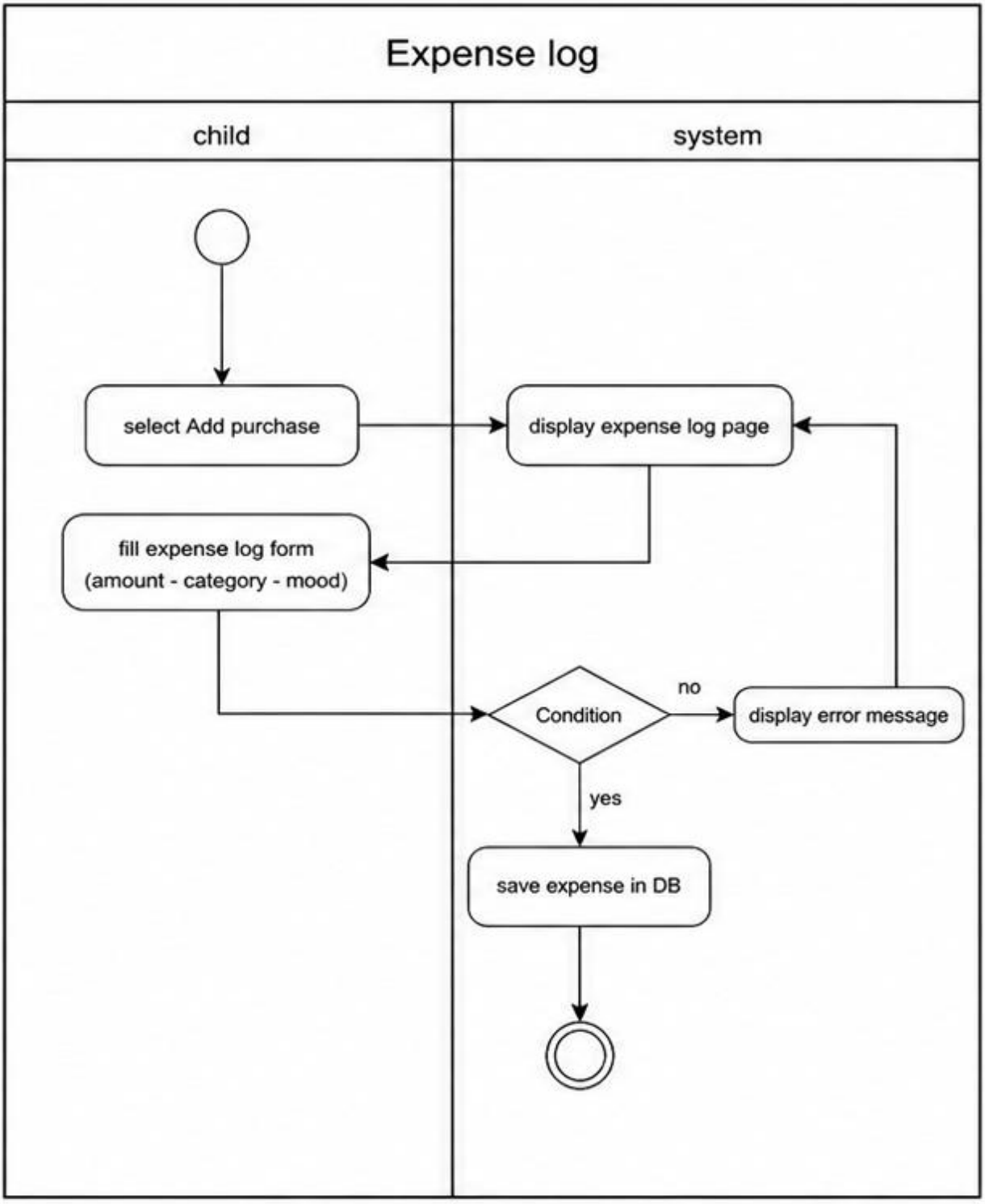

Multi-child Support
system
Parent
login to app
Retrieve children list from DB
Display sidebar listing children
Select a child from sidebar
Fetch selected child's data (expenses, profile, goals)
Display selected child's dashboard
Select another child (optional)
Fetch and display new child's data

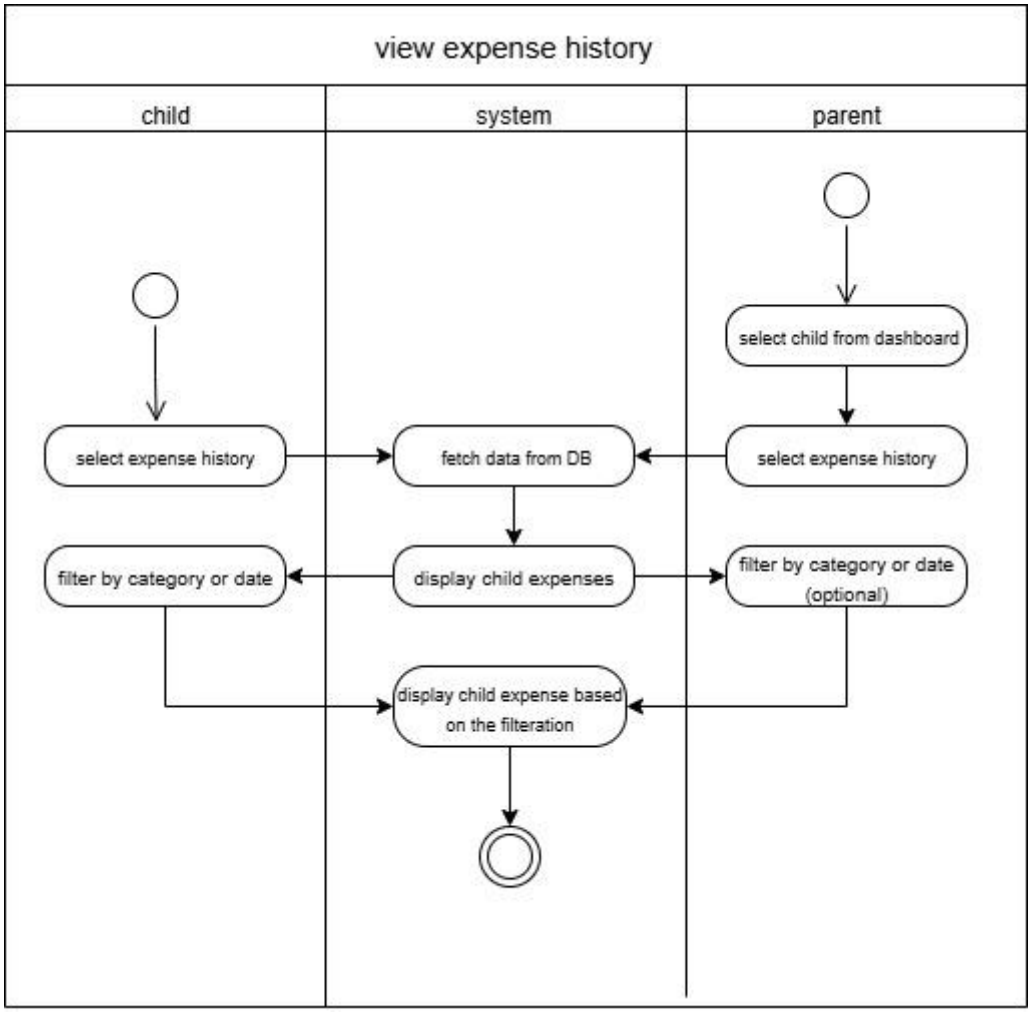

code generation and child login
child
system
parent
Open Add New Child
Display Add Child form
enter child data and relationship
answer questionnaire
Save child basic Info
Collect responses
Send response for AI service to calculate initial type
Analyze answers & return profile
Save profile in DB
generate and send child login code
receive login code
select login
display login form
enter login code
valid
no
display error message
yes
display home page

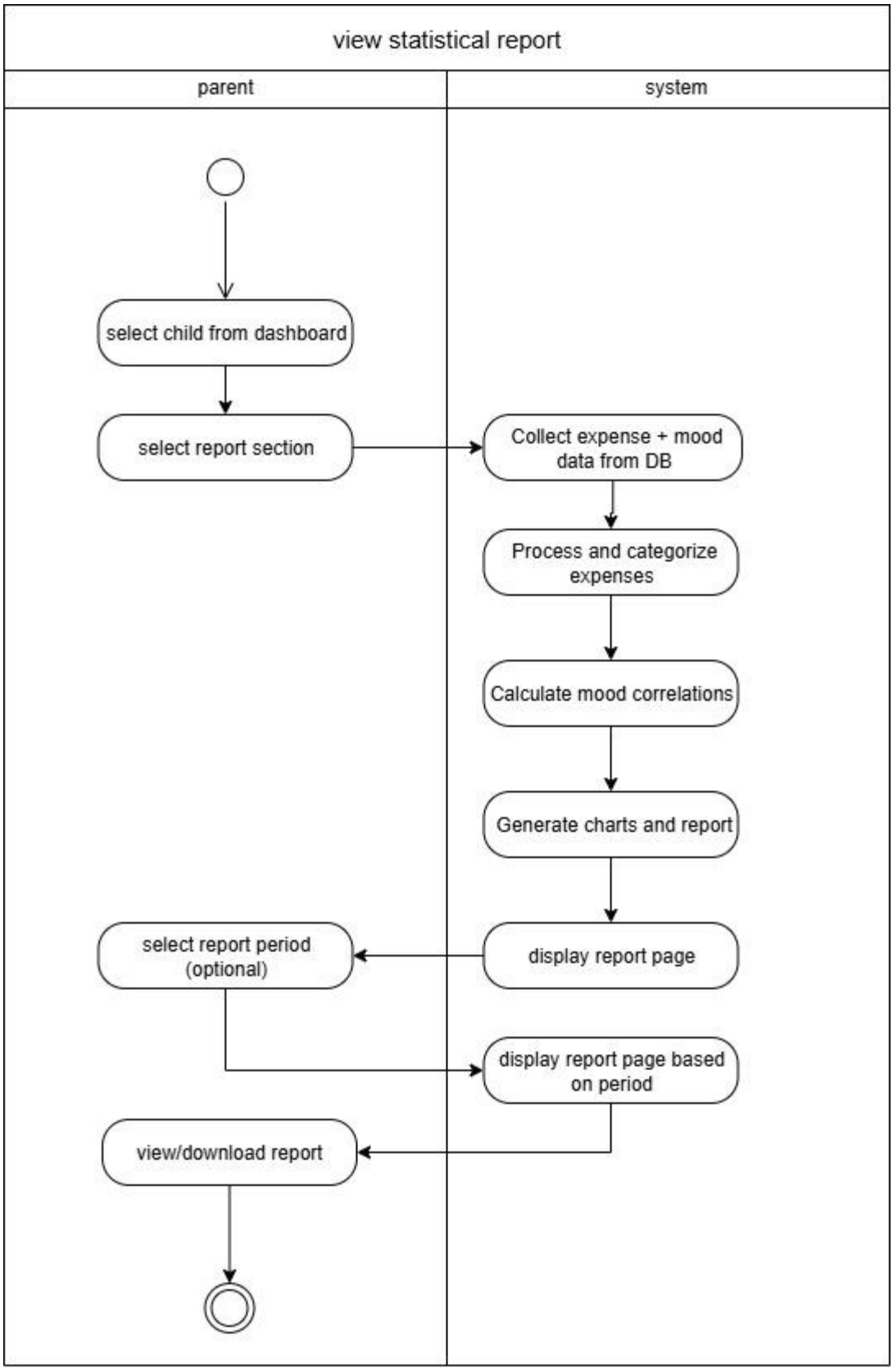

Allowance management
child
system
parent
fetch child data
select child
display home page with selected child key insight
select Allowance
Retrieve current and allowance History
display set allowance form
Enter weekly/monthly allowance and amount
Save new settings/bonus in DB
Add one-time bonus (optional)
Display updated allowance info in home page

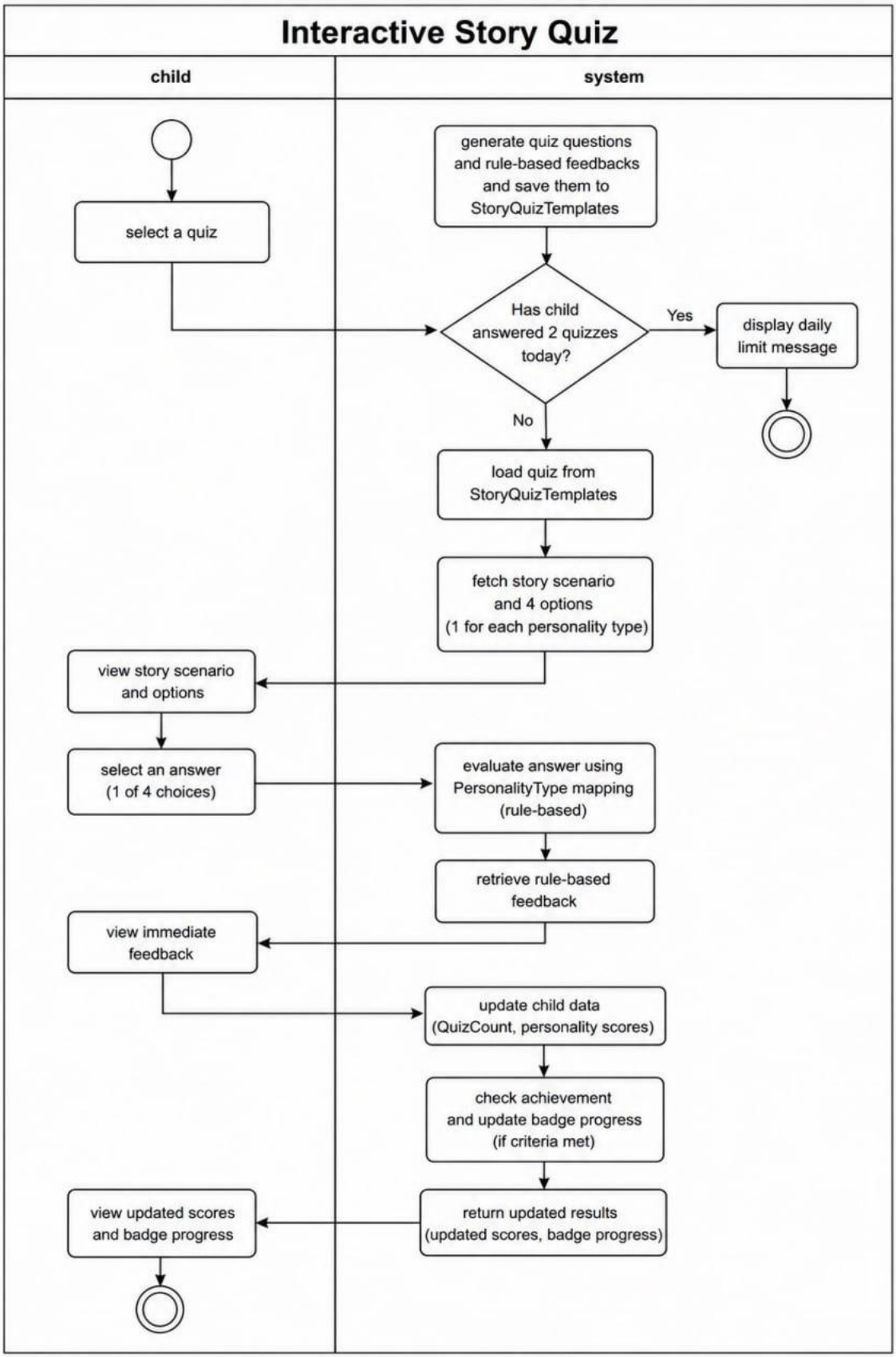

set goals by child
child
system
parent
select My Goals
display goal progress and parent's goal and achieved goals
fill goal form
display goal form
save goal to DB
display child goals

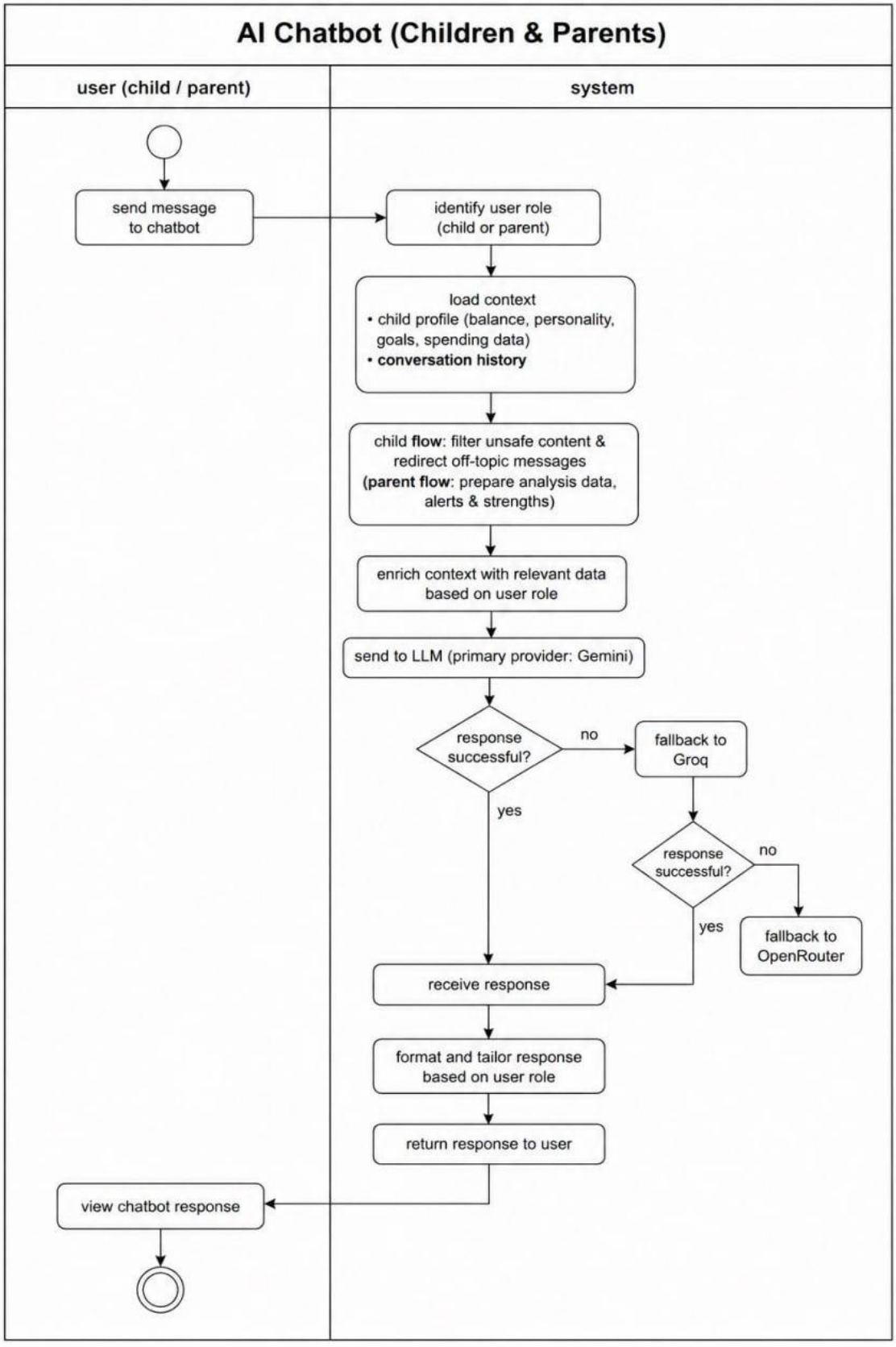

set goals by parent
child
system
parent
select child
display dashboard
select Goals
Retrieve challenge and saving goals from DB &and display
display goal form
fill goal info
save goal to DB
Notify the child
receive notification
select Goals
Retrieve goals and challenges
display all goals

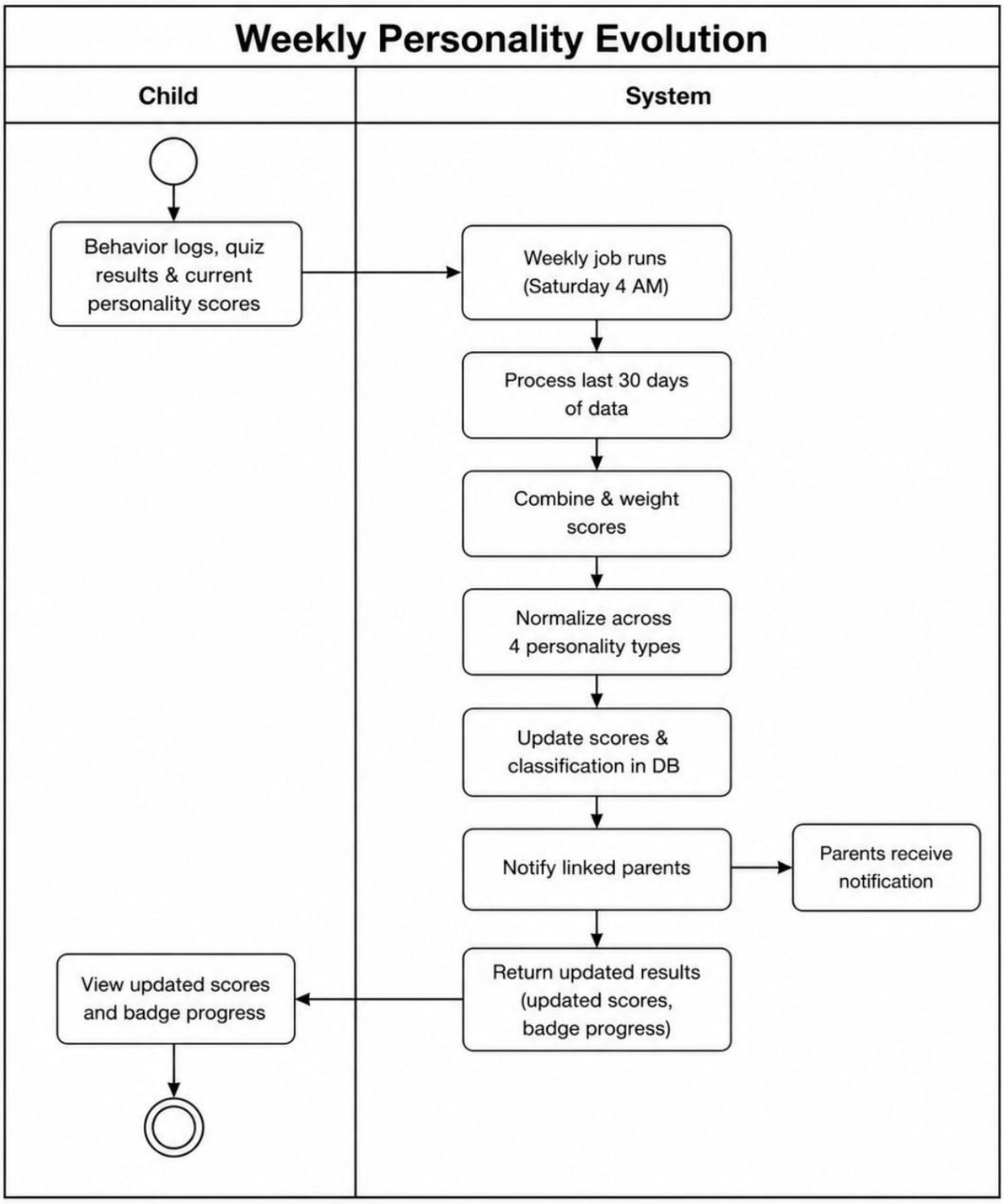

Expense log
child
system
select Add purchase
display expense log page
fill expense log form
(amount - category - mood)
Condition
no
display error message
yes
save expense in DB

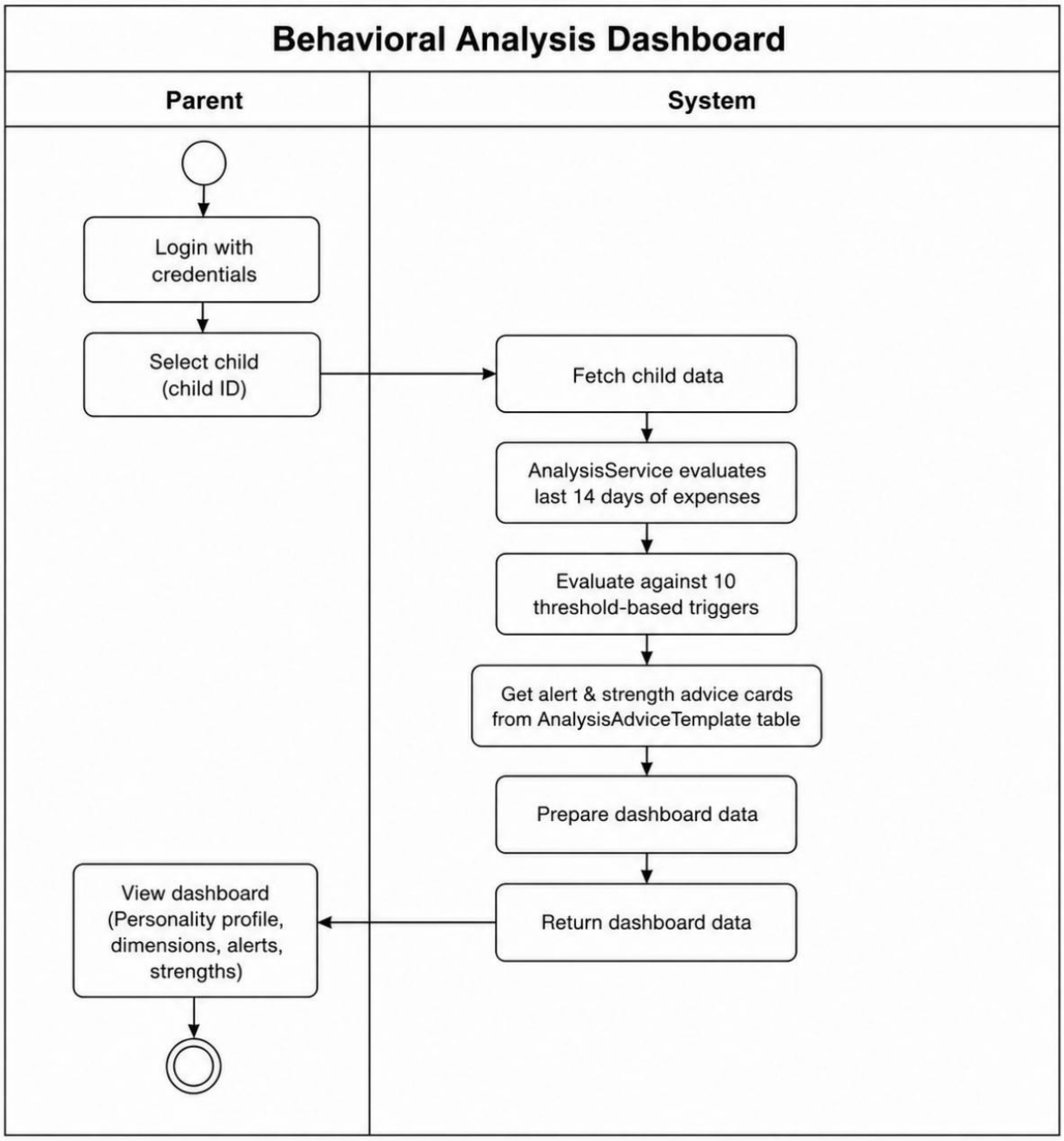

view expense history
child
system
parent
select child from dashboard
select expense history
fetch data from DB
select expense history
filter by category or date
display child expenses
filter by category or date (optional)
display child expense based on the filteration

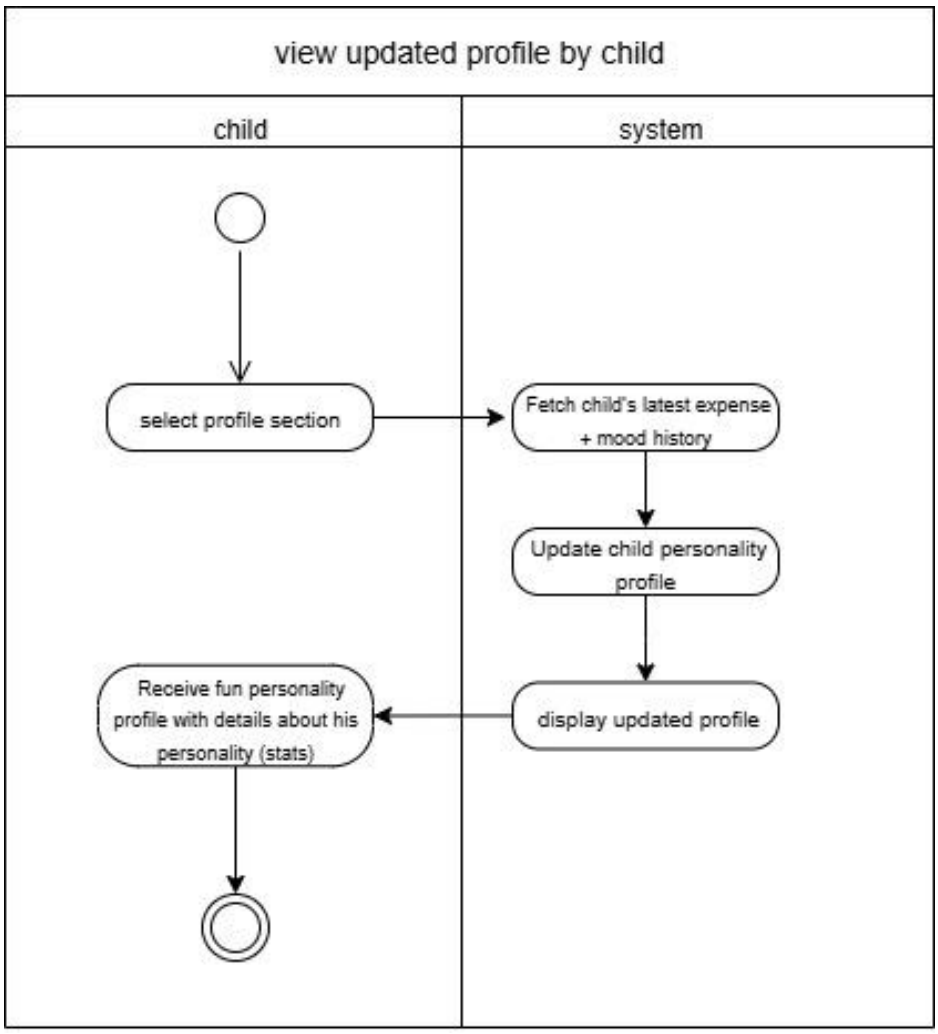

view statistical report
parent
system
select child from dashboard
select report section
Collect expense + mood data from DB
Process and categorize expenses
Calculate mood correlations
Generate charts and report
select report period (optional)
display report page
display report page based on period
view/download report

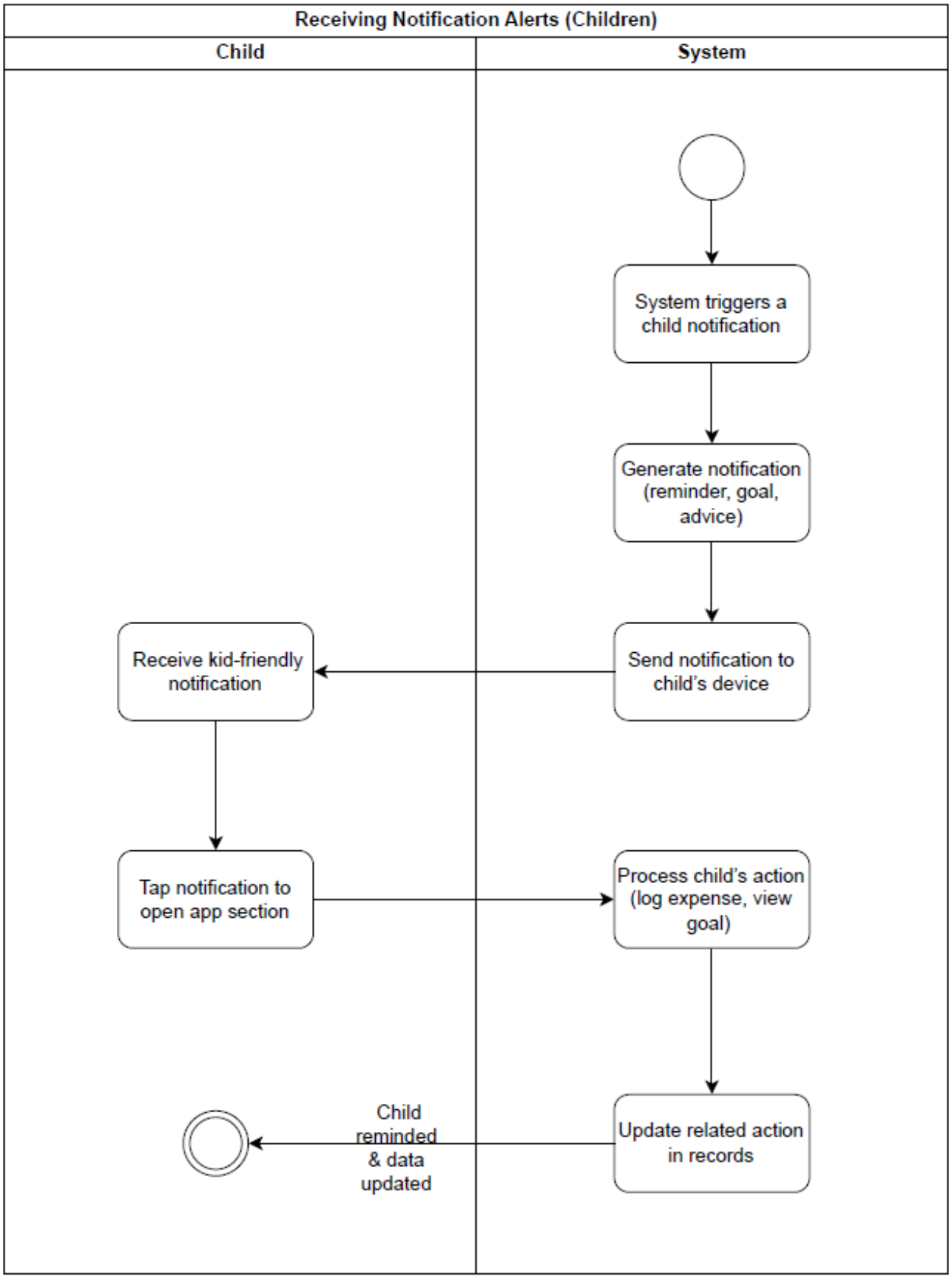

Interactive Story Quiz
child
system
select a quiz
generate quiz questions and rule-based feedbacks and save them to StoryQuizTemplates
Has child answered 2 quizzes today?
Yes
display daily limit message
No
load quiz from StoryQuizTemplates
fetch story scenario and 4 options (1 for each personality type)
view story scenario and options
select an answer (1 of 4 choices)
evaluate answer using PersonalityType mapping (rule-based)
retrieve rule-based feedback
view immediate feedback
update child data (QuizCount, personality scores)
check achievement and update badge progress (if criteria met)
view updated scores and badge progress
return updated results (updated scores, badge progress)

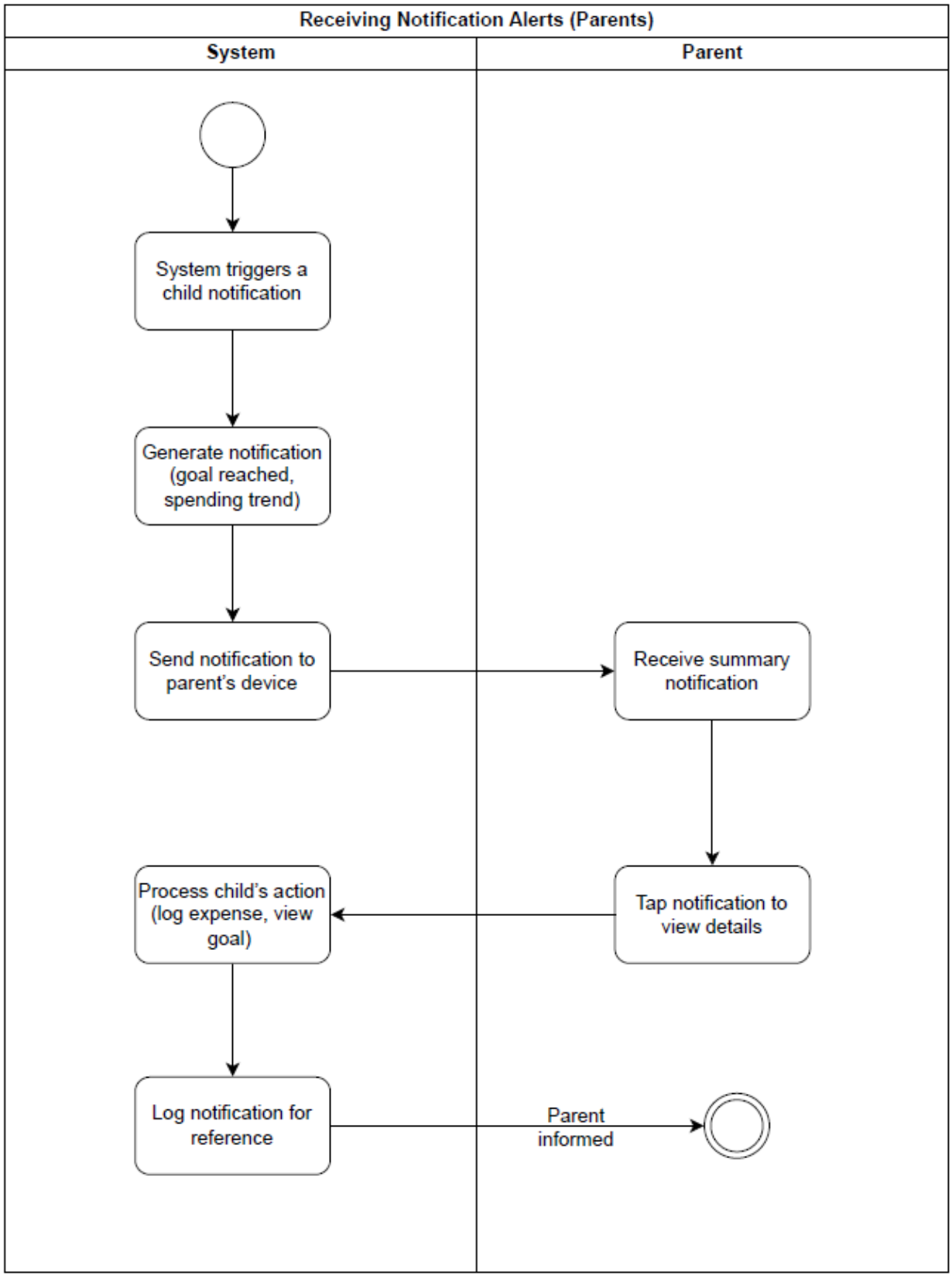

AI Chatbot (Children & Parents)
user (child / parent)
system
send message to chatbot
identify user role (child or parent)
load context
• child profile (balance, personality, goals, spending data)
• conversation history
child flow: filter unsafe content & redirect off-topic messages
(parent flow: prepare analysis data, alerts & strengths)
enrich context with relevant data based on user role
send to LLM (primary provider: Gemini)
response successful?
no
fallback to Groq
yes
response successful?
no
yes
fallback to OpenRouter
receive response
format and tailor response based on user role
return response to user
view chatbot response

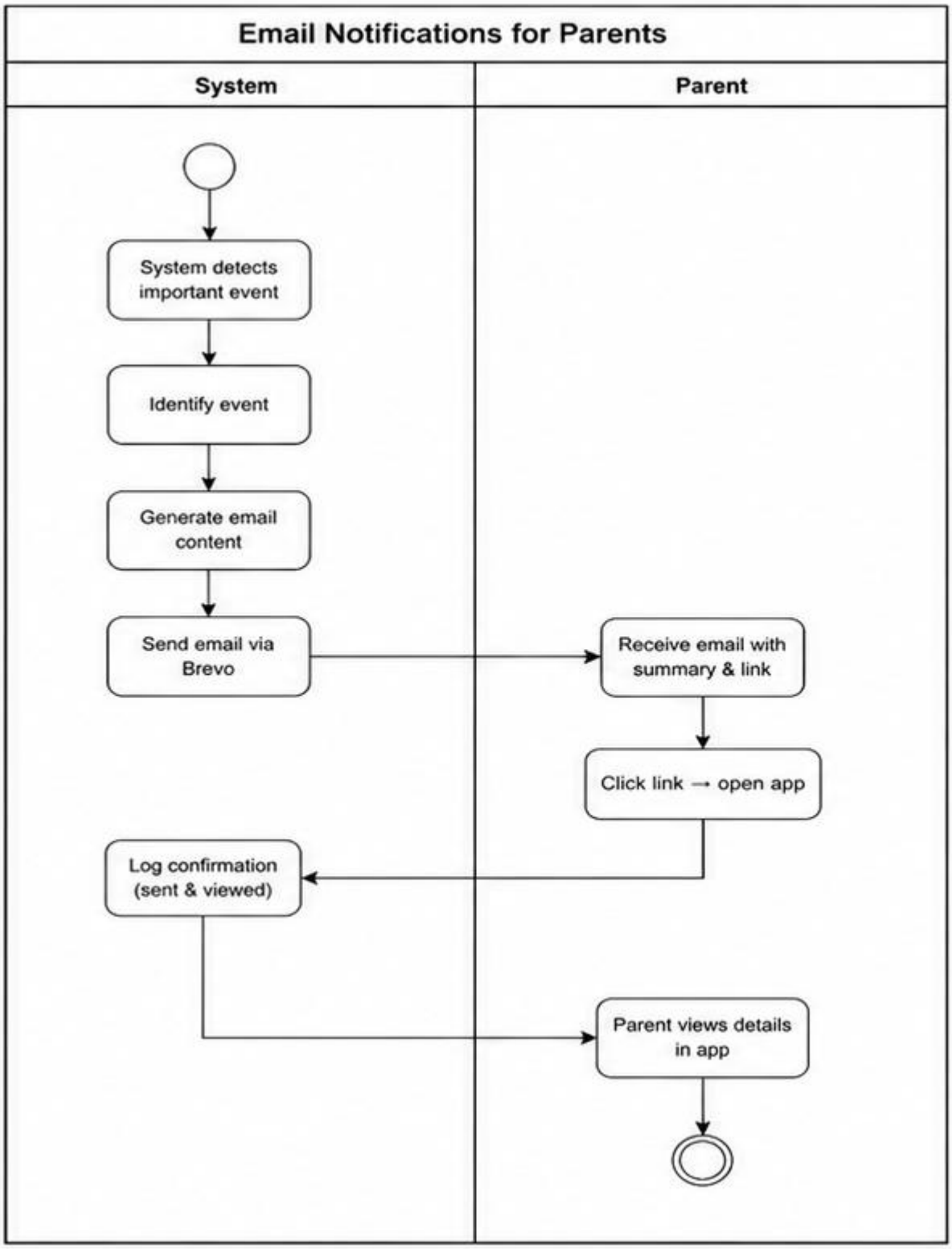

Weekly Personality Evolution
Child
System
Behavior logs, quiz results & current personality scores
Weekly job runs (Saturday 4 AM)
Process last 30 days of data
Combine & weight scores
Normalize across 4 personality types
Update scores & classification in DB
Notify linked parents
Parents receive notification
Return updated results (updated scores, badge progress)
View updated scores and badge progress

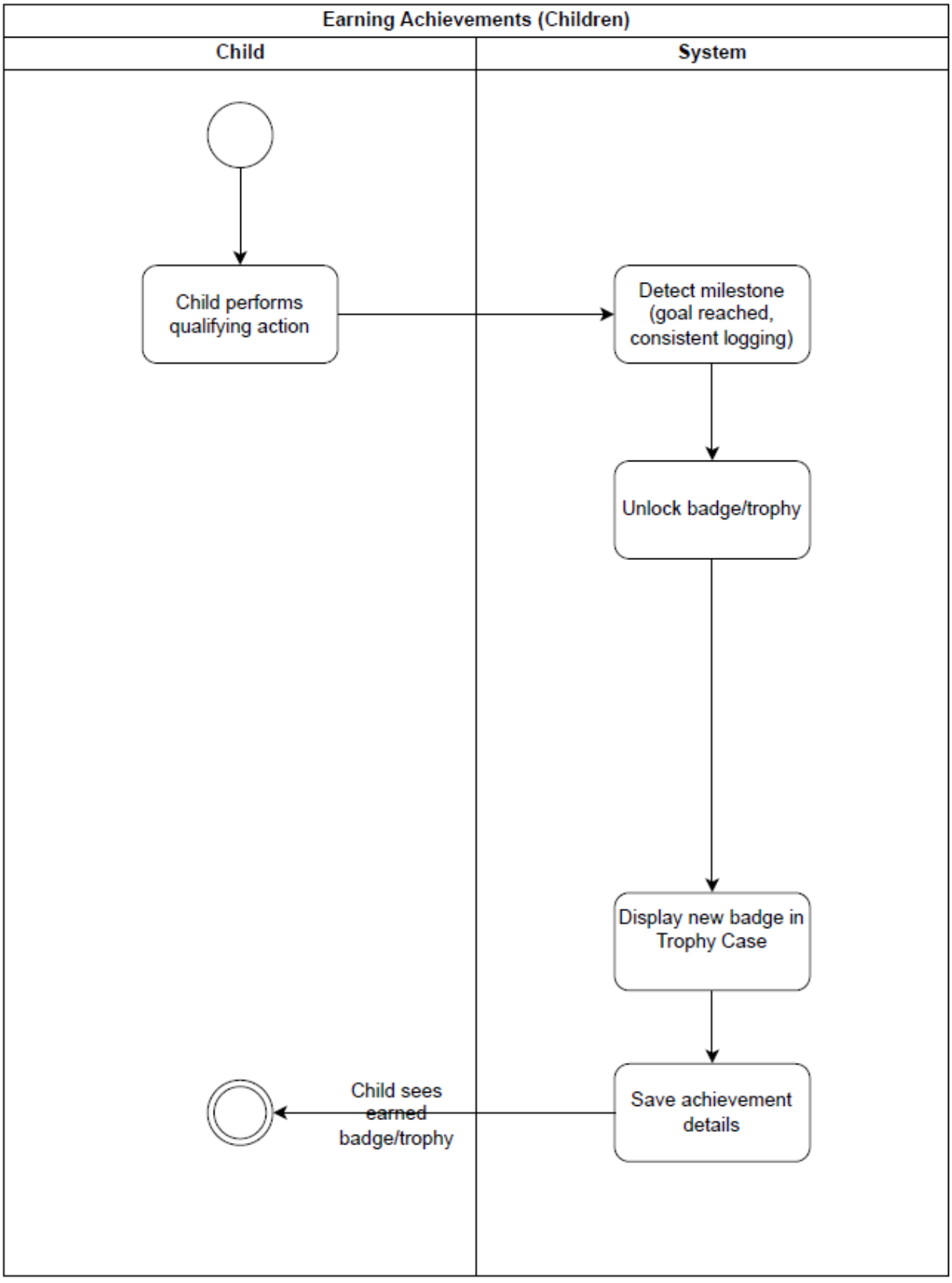

Behavioral Analysis Dashboard
Parent
System
Login with credentials
Select child (child ID)
Fetch child data
AnalysisService evaluates last 14 days of expenses
Evaluate against 10 threshold-based triggers
Get alert & strength advice cards from AnalysisAdviceTemplate table
Prepare dashboard data
Return dashboard data
View dashboard (Personality profile, dimensions, alerts, strengths)

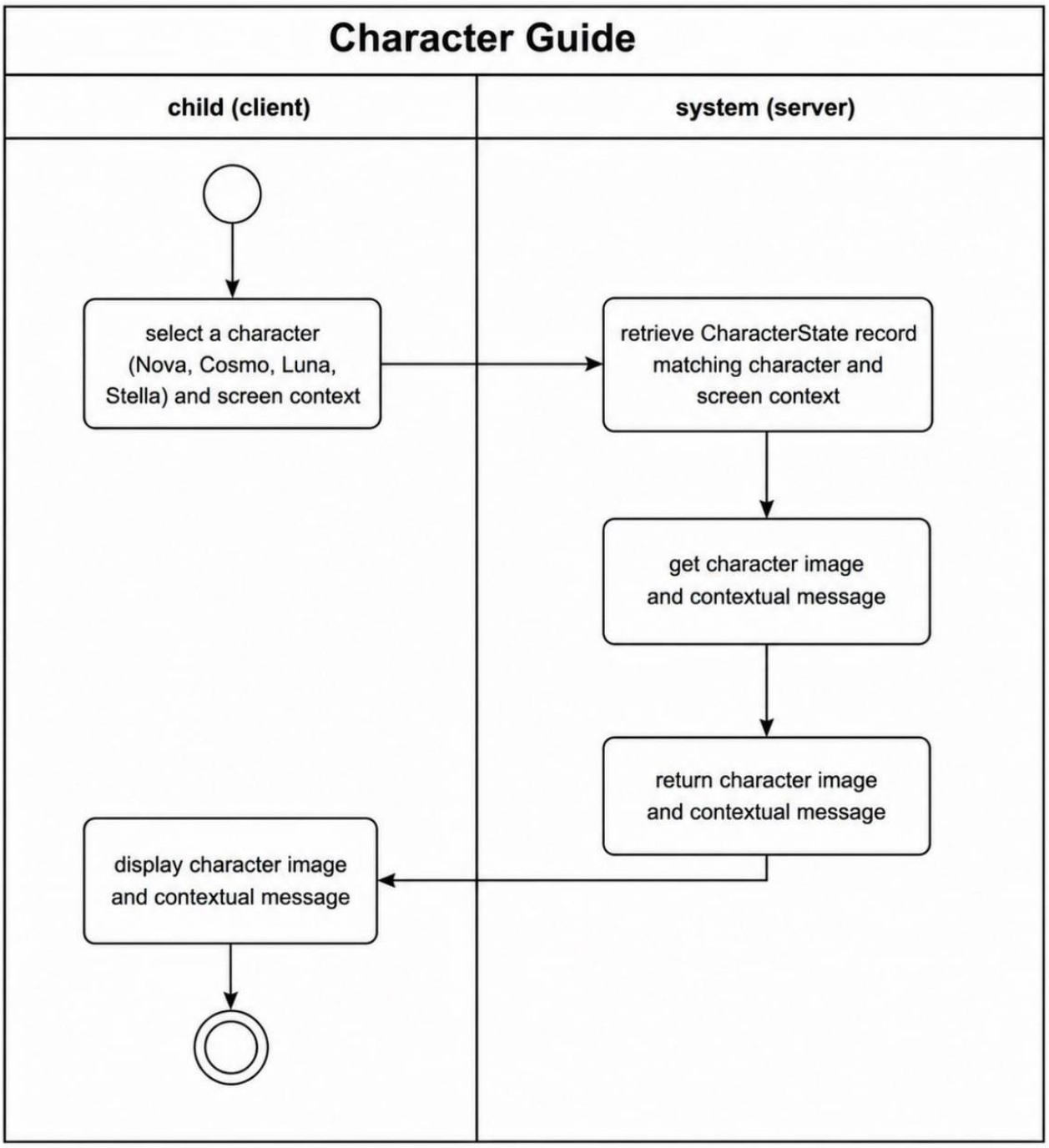

view updated profile by child
child
system
select profile section
Fetch child's latest expense + mood history
Update child personality profile
display updated profile
Receive fun personality profile with details about his personality (stats)


Receiving Notification Alerts (Children)
Child
System
System triggers a child notification
Generate notification (reminder, goal, advice)
Send notification to child's device
Receive kid-friendly notification
Tap notification to open app section
Process child's action (log expense, view goal)
Update related action in records
Child reminded & data updated


Receiving Notification Alerts (Parents)
System
Parent
System triggers a child notification
Generate notification (goal reached, spending trend)
Send notification to parent's device
Receive summary notification
Tap notification to view details
Process child's action (log expense, view goal)
Log notification for reference
Parent informed


Email Notifications for Parents
System
Parent
System detects important event
Identify event
Generate email content
Send email via Brevo
Receive email with summary & link
Click link → open app
Log confirmation (sent & viewed)
Parent views details in app


Earning Achievements (Children)
Child
System
Child performs qualifying action
Detect milestone (goal reached, consistent logging)
Unlock badge/trophy
Display new badge in Trophy Case
Save achievement details
Child sees earned badge/trophy


Character Guide
child (client)
system (server)
select a character (Nova, Cosmo, Luna, Stella) and screen context
retrieve CharacterState record matching character and screen context
get character image and contextual message
return character image and contextual message
display character image and contextual message

5. Feasibility Analysis

5.1 Executive Summary

Money Mirror is technically feasible using well-established, freely available technologies. The system avoids runtime LLM dependency for quiz serving and personality analysis, ensuring stability and fast response times. The personality analysis layer currently uses a deterministic rule-based engine, with a planned transition to trained ML models once sufficient behavioral data is collected. The target market—parents of children aged 6–14 seeking intelligent financial literacy tools—shows strong demand with limited AI-driven competition. Risk mitigations including phased development, offline batch processing for AI content, role-based security, and content validation pipelines are in place.

5.2 Technical Feasibility

Aspect	Details
Frontend	React Native — cross-platform (Android & iOS)
Backend	ASP.NET Core 8.0 Web API with JWT authentication and Hangfire scheduling
Database	SQL Server 2021 with Entity Framework Core 8.0
AI/Python Layer	Rule-based personality scoring, behavior analysis, and chatbot routing via Flask. LLM (Claude 3.5 Sonnet) for offline batch quiz generation only.
Chatbot LLMs	Gemini 2.5 Flash → Groq LLaMA-3.3-70B → OpenRouter DeepSeek (fallback chain)
Hardware (Mobile)	≥4 GB RAM, ≥1.5 GHz CPU, Android 8.0+ / iOS 11+.
Hardware (Backend)	Standard server (≥8 GB RAM, ≥4 vCPU).
Expertise Needed	Team has experience in React Native, C#/.NET, SQL Server, and Python

5.3 Market Feasibility

5.3.1 Target Market Analysis

- Primary: Parents/guardians of children aged 6-14 (~300M households globally, ~15M in MENA)
- Pain points: No visibility into children's spending, difficulty teaching financial concepts engagingly, lack of personality-aware guidance

5.3.2 Competitors Landscape

Competitor	Core Features	Gaps vs. Money Mirror
Greenlight	Prepaid card + chores	No AI personality insights, limited gamification
FamZoo	Family banking, IOUs	Rule-based only, no mood correlation
Bankaroo	Virtual bank for kids	Basic categorization, no behavioral analytics
GoHenry	Debit card + tasks	Transactional focus, minimal learning modules

5.3.3 Money Mirror Differentiation:

- Personality-based behavioral profiling with evolving weekly analysis
- Mood-spending correlation insights
- Gamified story-based financial quizzes
- Dual-interface design (detailed parent analytics + engaging child experience)
- AI chatbot coach with full child context awareness

5.4 Risk Analysis

Risk	Level	Type	Mitigation
Rule-based AI integration complexity	High	Technical	Phased development; incremental testing; well-documented libraries
Data breach / unauthorized access	High	Security	JWT auth, encrypted DB, role-based access control, COPPA/GDPR compliance
Quiz content integrity	Medium	Content	Strict validation pipeline: 4 choices, all personality types, no duplicates, manual review of batches
System performance under load	Medium	Performance	Pre-generated quiz content (no runtime LLM), DB query optimization, stress testing
Low user adoption	High	Market	Pilot programs with schools; iterative UX improvements based on feedback
Child data protection compliance	Medium	Legal	Legal review of all privacy policies; full COPPA and GDPR compliance
LLM chatbot provider outage	Low	Technical	Multi-provider fallback chain: Gemini → Groq → OpenRouter

6. Nonfunctional Requirements

6.1 Performance Requirements

- Child dashboard (balance, goals, achievements): loads within 2 seconds under normal network conditions
- Parent analytics reports (spending breakdown, mood correlation): generated within 3 seconds
- Chatbot response time: within 10 seconds (live LLM call with fallback)
- Quiz loading: instant (database retrieval, no LLM call)
- Personality score updates: under 2 seconds
- Weekly personality update: Hangfire background job only, no user-facing latency
- System supports multiple parent-child accounts concurrently without degradation

6.2 Safety Requirements

- Parents have full control over data visibility; children see only simplified, age-appropriate views
- Daily backups of all financial logs, personality profiles, and behavioral data
- No real-time LLM exposure to children; all child-facing content is pre-approved or rule-governed
- All quiz content validated before database insertion; content filtering applied
- Child interface restricted exclusively to safe, pre-approved data

6.3 Security Requirements

- All API endpoints protected by JWT Bearer authentication
- Role-based access control: Parent role vs. Child role — children cannot access parent dashboards
- Parent accounts require strong password (8+ chars, uppercase, lowercase, digit, special character)
- Sensitive data (allowance history, personality profiles) encrypted in SQL Server
- Python Flask AI service isolated from frontend; not directly accessible by clients
- Refresh tokens stored hashed (BCrypt) in database
- 6-digit time-limited codes for email confirmation, password reset, and email change
- Soft delete for parent accounts with 30-day recovery grace period

6.4 Software Quality Attributes

- **Correctness**: Financial logs and analytics must reflect accurate values as entered or calculated
- **Usability**: Child UI: colorful, simple, emoji-driven. Parent UI: data-rich, filterable, informative
- **Reliability**: Auto-save on all financial operations; no data loss on crash; transaction rollback on failure
- **Maintainability**: Standard ASP.NET Core and React Native conventions; interface/service architecture for easy extension
- **AI Stability**: No runtime LLM dependency for core features; chatbot has multi-provider fallback chain.
- **Scalability**: Supports 2+ children per parent; designed for incremental quiz content expansion and new personality types.
- **Testability**: Unit and integration tests for expense logging, personality scoring, allowance crediting, and report generation.
- **Robustness**: Graceful handling of invalid inputs (amounts, dates, codes); informative error messages returned via ApiResponse wrapper
- **Extensibility**: Architecture supports future: additional personality types, premium analytics tiers, expanded quiz modules, ML model integration.
- **Performance Efficiency**: Pre-generated quiz content ensures instant load; Hangfire offloads scheduled work from request threads; indexed database queries for high-frequency read paths.
- **Monetization Readiness**: System architecture supports future paid features (premium analytics dashboards, advanced insights, enhanced quiz content) while keeping core features free.